



**Tarlac State University**  
**College of Computer Studies**  
**Bachelor of Science in Computer Science**  
**Operating Systems**



**CPU Process Scheduling Algorithms**

A Project

Submitted to the Faculty of the College of Computer Studies

San Isidro Campus, Tarlac City,

Tarlac State University

In Partial Fulfillment of the Requirements for the Course Operating Systems

Academic Year 2025-2026

Submitted By:

**Franco, King Paolo D.**

Subject Instructor:

**Jo Anne G. Cura**

Date of submission:

**May 15, 2026**

# Table of Contents

- 1. Introduction..... 6
  - 1.1. Project Overview ..... 6
  - 1.2. Project Purpose ..... 7
  - 1.3. Project Objectives ..... 7
- 2. Body..... 9
  - 2.1. First Come, First Served ..... 9
    - 2.1.1. How It Works ..... 9
    - 2.1.2. Preemptive or non-preemptive..... 9
    - 2.1.3. Advantages and Limitations ..... 9
    - 2.1.4. Test Cases..... 9
  - 2.2. Shortest Job First..... 10
    - 2.2.1. How It Works ..... 11
    - 2.2.2. Preemptive or non-preemptive..... 11
    - 2.2.3. Advantages and Limitations ..... 11
    - 2.2.4. Test Cases..... 11
  - 2.3. Shortest Remaining Time..... 12
    - 2.3.1. How It Works ..... 13
    - 2.3.2. Preemptive or non-preemptive..... 13
    - 2.3.3. Advantages and Limitations ..... 13
    - 2.3.4. Test Cases..... 13
  - 2.4. Round Robin ..... 15
    - 2.4.1. How It Works ..... 15
    - 2.4.2. Preemptive or non-preemptive..... 15
    - 2.4.3. Advantages and Limitations ..... 15
    - 2.4.4. Test Cases..... 15
  - 2.5. Priority Scheduling ..... 17
    - 2.5.1. How It Works ..... 17
    - 2.5.2. Preemptive or non-preemptive..... 17
    - 2.5.3. Advantages and Limitations ..... 17
    - 2.5.4. Test Cases..... 18
  - 2.6. Priority Scheduling with Round Robin..... 19
    - 2.6.1. How It Works ..... 19
    - 2.6.2. Preemptive or non-preemptive..... 20
    - 2.6.3. Advantages and Limitations ..... 20
    - 2.6.4. Test Cases..... 20

- 3. Summary ..... 23
  - 3.1. Key Insights ..... 23
  - 3.2. Performance Differences Observed ..... 23
  - 3.3. Practical Relevance in Modern OS ..... 24
  - 3.4. Challenges & How They Were Addressed..... 24
  - 3.5. Cross-Algorithm Comparison ..... 25
- 4. References ..... 26
- 5. Repository Link ..... 28
- 6. Source Code ..... 29

# List of Figures

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| Figure 1. FCFS - Test Case #1 .....                                                  | 10 |
| Figure 2. FCFS - Test Case #2 .....                                                  | 10 |
| Figure 3. SJF Test Case #1 .....                                                     | 12 |
| Figure 4. SJF Test Case #2.....                                                      | 12 |
| Figure 5. SRT Test Case #1 .....                                                     | 14 |
| Figure 6. SRT Test Case #2.....                                                      | 14 |
| Figure 7. RR Test Case #1 .....                                                      | 16 |
| Figure 8. RR Test Case #2 .....                                                      | 16 |
| Figure 9. Priority Test Case #1 Lower Value = Higher Priority & Non-preemptive ..... | 18 |
| Figure 10. Priority Test Case #2 Higher Value = Higher Priority & Preemptive .....   | 19 |
| Figure 11. Priority + RR Test Case #1 Lower Value = Higher Priority .....            | 21 |
| Figure 12. Priority + RR Test Case #2 Higher Value = Higher Priority .....           | 22 |

List of Tables

Table 3. FCFS Test Case #1 ..... 9

Table 4. FCFS Test Case #2 ..... 10

Table 5. SJF Test Case #1 ..... 11

Table 6. SJF Test Case #2 ..... 12

Table 7. SRT Test Case #1 ..... 14

Table 8. SRT Test Case #2 ..... 14

Table 9. RR Test Case #1 ..... 15

Table 10. RR Test Case #2 ..... 16

Table 11. Priority Test Case #1 Lower Value = Higher Priority & Non-preemptive ..... 18

Table 12. Priority Test Case #2 Higher Value = Higher Priority & Preemptive ..... 18

Table 13. Priority + RR Test Case #1 Lower Value = Higher Priority ..... 20

Table 14. Priority + RR Test Case #2 Higher Value = Higher Priority ..... 21

# 1. Introduction

## 1.1. Project Overview

CPU scheduling is a fundamental part of operating systems, responsible for deciding the order in which processes access the CPU. By prioritizing tasks, CPU scheduling helps manage system performance and responsiveness, ensuring that each active process gets its fair share of the CPU's resources. This is especially important in multitasking environments, where multiple programs run concurrently (Goyal, 2024).

The operating system increases productivity by switching among CPU processes. A process executes until it typically waits for some I/O request, meaning the CPU would otherwise sit idle. With multiprogramming, the operating system aims to use this idle time productively by keeping several processes in memory at once — when one process must wait, the OS assigns the CPU to another. Scheduling is therefore a fundamental operating system function, as almost all computer resources are scheduled before use, making CPU scheduling central to operating system design (Shandilya, 2024).

CPU scheduling governs the sharing of processor time among numerous tasks running on a computer, ensuring efficiency and fairness in executing processes, and that the system can fulfill its users' performance and responsiveness requirements (Peter, 2023).

The primary goals that CPU scheduling aims to achieve include maximizing CPU utilization to keep the processor as busy as possible and minimizing both the response time and waiting time experienced by each process (GeeksforGeeks, 2026).

### Key Terminologies in CPU Scheduling

Understanding the following terms is essential for analyzing and comparing scheduling algorithms (GeeksforGeeks, 2026; Peter, 2023):

- **Arrival Time** - The time at which a process enters the ready queue, awaiting CPU execution.
- **Burst Time** - The total amount of CPU time a process requires to complete its execution.
- **Completion Time** - The point in time at which a process finishes its execution entirely.
- **Turnaround Time (TAT)** - The total elapsed time from a process's arrival to its completion.
  - $Turnaround\ Time = Completion\ Time - Arrival\ Time$
- **Waiting Time (WT)** - The total time a process spends waiting in the ready queue before receiving CPU time.

$$\circ \text{ Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$$

- **Response Time** - In an interactive system, response time is the time taken from the submission of a process until its first response is issued, rather than its full completion.

## Types of CPU Scheduling

CPU scheduling is broadly classified into two types (TutorialsPoint, n.d.):

- **Non-preemptive Scheduling** - A resource cannot be taken from a process until that process completes execution. The switching of resources occurs only when the running process terminates or moves to a waiting state.
- **Preemptive Scheduling** - The OS allocates resources to a process for a fixed amount of time. During this allocation, a process may switch from a running state to a ready state, as the CPU may give priority to higher-priority processes and replace the currently running one.

## 1.2. Project Purpose

The study of CPU scheduling algorithms is a cornerstone of any Operating Systems course yet understanding them through theory alone often leaves gaps in comprehension. In the teaching of operating systems, which is part of computer engineering and computer science degrees, a thorough understanding of processor scheduling algorithms is crucial. However, abstract descriptions of how these algorithms behave are insufficient without a practical, hands-on counterpart to reinforce those concepts (González-Rodríguez et al., 2024).

This case study addresses that gap by implementing a working simulation of six fundamental CPU scheduling algorithms: First-Come, First-Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time (SRT), Round Robin (RR), Priority Scheduling, and Priority Scheduling with Round Robin. A scheduling simulator illustrates the behavior of scheduling algorithms against a simulated mix of process loads; it is a framework that allows one to swiftly and easily devise and collect metrics for CPU scheduling strategies. To calculate the comparative and competitive advantages and disadvantages of these algorithms, each algorithm must be simulated and their performance indices studied for better capturing of operating system principles.

Through this simulation, the program accepts real process parameters arrival times, burst times, priorities, and time quantum, and produces measurable outputs such as Gantt charts, waiting times, and turnaround times. Such evaluation results are extremely useful for the design and development of modern operating systems, and a simulator of this kind can be used for measuring the performance of different scheduling algorithms.

## 1.3. Project Objectives

This case study aims to design and develop a web-based CPU scheduling simulator that demonstrates the practical behavior of fundamental scheduling algorithms used in operating systems. Specifically, this project seeks to achieve the following objectives:

1. To implement a functional web-based CPU scheduling simulator that accepts user-defined process inputs, supporting a minimum of three (3) and a maximum of ten (10)

processes per simulation, with each process defined by its process identifier, arrival time, burst time, and priority value.

2. To simulate six (6) CPU scheduling algorithms within the application, namely: First-Come, First-Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time (SRT), Round Robin (RR), Priority Scheduling, and Priority Scheduling with Round Robin, allowing the user to select and switch between algorithms freely.
3. To generate a visual Gantt Chart for each simulation run, accurately representing the execution order and timeline of all processes under the selected scheduling algorithm, including preemption points where applicable.
4. To display a comprehensive process summary table for each simulation, presenting per-process values for Arrival Time (AT), Burst Time (BT), Completion Time (CT), Waiting Time (WT), and Turnaround Time (TAT), alongside the computed Average Waiting Time and Average Turnaround Time across all processes.
5. To provide an interactive and accessible learning tool through a web interface, enabling users to observe and compare how different scheduling decisions affect process performance metrics without requiring any software installation.



## 2. Body

### 2.1. First Come, First Served

#### 2.1.1. How It Works

First-Come, First-Served is the simplest CPU scheduling algorithm. As the name implies, the process that arrives first in the ready queue is the first to be allocated CPU time. Processes are executed in the exact order of their arrival, and each process runs to completion before the next one begins. The ready queue in FCFS operates as a standard FIFO (First-In, First-Out) queue, where the CPU is assigned to the process at the front of the queue and is not released until that process finishes its burst time entirely (Silberschatz et al., 2018).

When a new process arrives while the CPU is busy, it simply joins the back of the ready queue and waits. No reordering or prioritization occurs; the arrival order is the sole determining factor for execution sequence. This straightforward nature makes FCFS easy to understand and implement, but it also introduces a key behavioral characteristic: shorter processes that arrive after a long process are forced to wait for the entirety of that long process to complete, regardless of how brief their own burst times are (GeeksforGeeks, 2026).

#### 2.1.2. Preemptive or non-preemptive

FCFS is a non-preemptive scheduling algorithm. Once a process is assigned to the CPU, it holds the CPU until it completes its execution or voluntarily enters a waiting state. No other process can interrupt or displace a currently running process under this algorithm.

#### 2.1.3. Advantages and Limitations

**Advantages:**

- Simple and easy to implement due to its straightforward FIFO structure
- Fair in the sense that every process is eventually served in the order it arrives, with no indefinite postponement or starvation
- Low scheduling overhead, as no complex comparisons or reordering of the queue is needed at any point

**Limitations:**

- Suffers from the Convoy Effect, where shorter processes are forced to wait behind a long-running process, resulting in poor average waiting times for the entire process set (Silberschatz et al., 2018)
- Not suitable for time-sharing or interactive systems where quick response times are critical
- Does not consider burst time or priority, making it inefficient when process lengths vary significantly

#### 2.1.4. Test Cases

**Test Case 1**

Table 1. FCFS Test Case #1

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 8          |
| P2      | 1            | 4          |
| P3      | 2            | 2          |

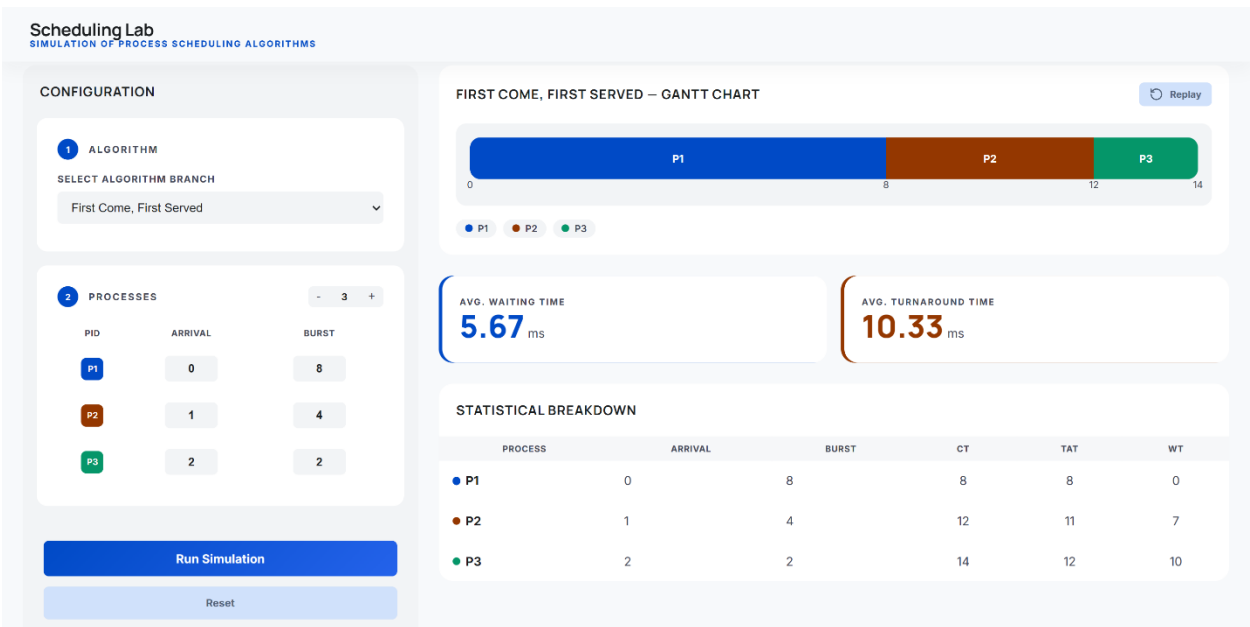


Figure 1. FCFS - Test Case #1

This test case clearly demonstrates the **Convoy Effect** - P3 has only 2 units of burst time but waits 10 units because it is stuck behind P1 and P2.

Test Case 2

Table 2. FCFS Test Case #2

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 3          |
| P2      | 1            | 6          |
| P3      | 4            | 2          |
| P4      | 5            | 1          |

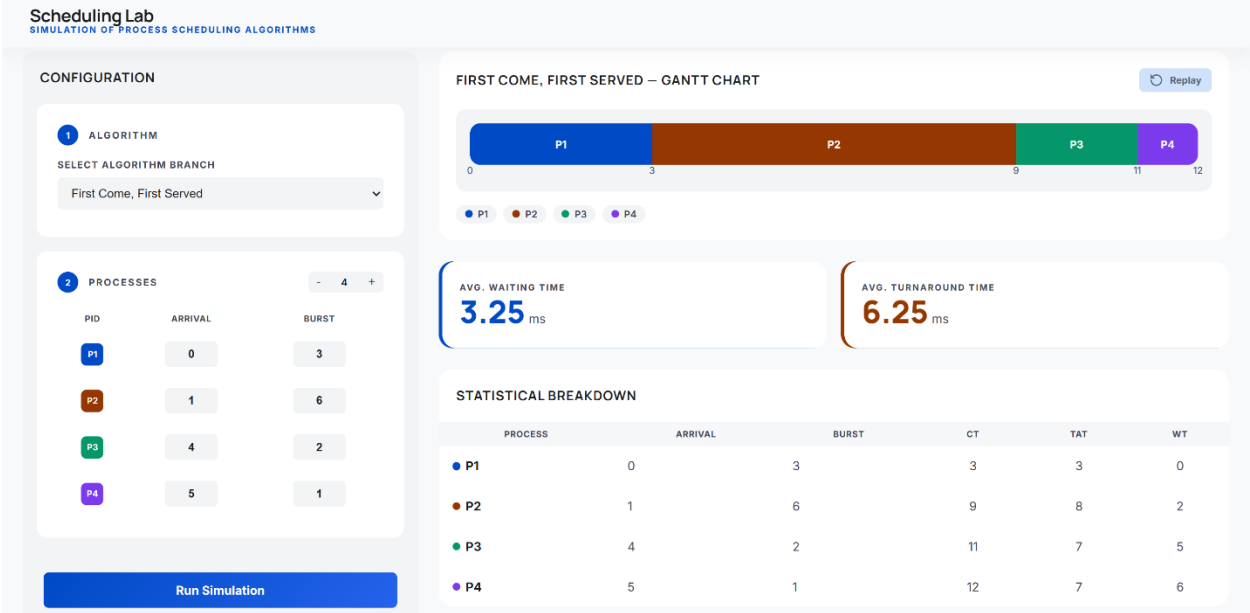


Figure 2. FCFS - Test Case #2

This test case introduces a gap - P3 arrives at time 4 but P2 is still running, illustrating how FCFS handles staggered arrivals without any reordering.

2.2. Shortest Job First

2.2.1. How It Works

Shortest Job First is a scheduling policy that selects the waiting process with the smallest execution time to execute next. It is considered a greedy algorithm, as it always picks the locally optimal choice at each scheduling decision point. When the CPU becomes free, the scheduler looks at all processes currently in the ready queue and selects the one with the shortest burst time. If two processes have the same burst time, a tie-breaking rule such as FCFS order is applied (GeeksforGeeks, 2026).

Unlike FCFS, which serves processes strictly by arrival order, SJF reorders the ready queue based on burst time, ensuring that shorter processes are not unnecessarily delayed behind longer ones. SJF significantly reduces the average waiting time for other processes awaiting execution and is particularly helpful for batch-type processing, where waiting for jobs to complete is not critical (GeeksforGeeks, 2026).

2.2.2. Preemptive or non-preemptive

SJF as implemented in this simulator is non-preemptive. Once the CPU is assigned to the process with the shortest burst time, that process runs to completion without interruption, even if a new process with a shorter burst time arrives during execution.

2.2.3. Advantages and Limitations

Advantages:

- SJF has the shortest average waiting time out of all scheduling methods and is probably the best in terms of average turnaround time (Marthur, 2022).
- Performs better than FCFS in most scenarios involving processes of varying burst lengths, as shorter processes are not blocked by longer ones
- Simple to implement in batch processing environments where burst times are known ahead of time

Limitations:

- SJF has the potential for process starvation, processes that require a long time to complete may wait indefinitely if shorter processes are continually added to the ready queue (Wadmare et al., 2024).
- Predicting the upcoming CPU request is difficult, and the algorithm generates starvation or extremely long turnaround times for longer processes.
- Does not account for process priority or urgency, meaning a critical but long process may be repeatedly passed over

2.2.4. Test Cases

Test Case 1

Table 3. SJF Test Case #1

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 7          |
| P2      | 2            | 4          |
| P3      | 4            | 1          |

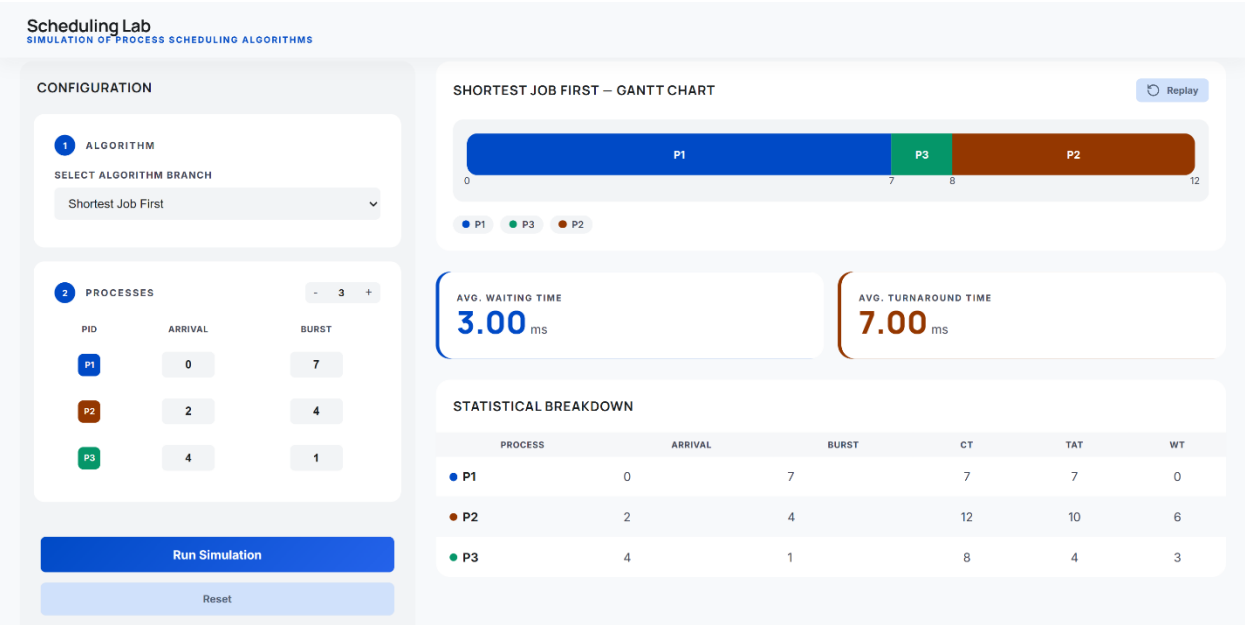


Figure 3. SJF Test Case #1

P1 starts immediately at time 0 since it is the only available process. By time 7, both P2 and P3 have arrived. The scheduler selects P3 first due to its shorter burst time of 1, then P2 last.

Test Case 2

Table 4. SJF Test Case #2

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 5          |
| P2      | 1            | 3          |
| P3      | 2            | 6          |
| P4      | 3            | 2          |

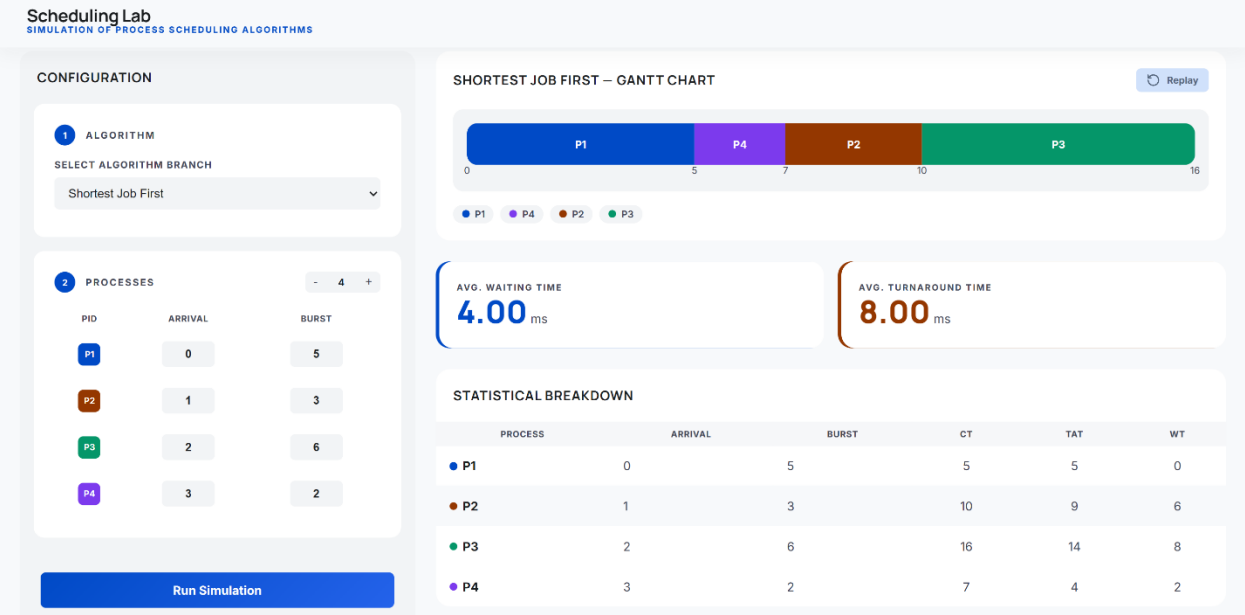


Figure 4. SJF Test Case #2

P1 is the only available process at time 0. After it finishes at time 5, the scheduler evaluates P2, P3, and P4 together and picks P4 (burst = 2) first, then P2 (burst = 3), and finally P3 (burst = 6) last.

2.3.Shortest Remaining Time

### 2.3.1. How It Works

Shortest Remaining Time (SRT), also known as Shortest Remaining Time First (SRTF), is a scheduling method that is the preemptive version of Shortest Job Next scheduling. In this scheduling algorithm, the process with the smallest amount of time remaining until completion is selected to execute (Marthur, 2022).

At every point a new process arrives in the ready queue, the operating system compares the remaining burst time of the currently running process against the burst time of the newly arrived process. The processor is allocated to the job closest to completion, but even this job can be preempted if a newer job in the ready queue has a time to completion that is shorter. When a preemption occurs, the currently running process is moved back into the ready queue with its updated remaining burst time, and the new shorter process takes control of the CPU (Buck & Perugini, 2020).

SRT involves more overhead than Shortest Job First (SJF), as the operating system is required to frequently monitor the CPU time of the jobs in the ready queue and perform context switching. Once all processes have arrived and no new arrivals can trigger a preemption, the algorithm behaves similarly to non-preemptive SJF, continuing to execute whichever process in the queue has the least remaining time until all processes complete (Saleem & Javed, 2026).

### 2.3.2. Preemptive or non-preemptive

SRT is a preemptive scheduling algorithm. The currently executing process can be interrupted and displaced at any moment a newly arriving process has a shorter remaining burst time, making it the most responsive among the shortest-job-based scheduling strategies.

### 2.3.3. Advantages and Limitations

#### Advantages:

- SRT achieves the minimum possible average waiting time among all scheduling algorithms when burst times are known, as it always prioritizes the process closest to completion.
- It is more responsive than non-preemptive SJF because a newly arrived short process can immediately displace a longer running one, rather than waiting for it to finish entirely.
- Short processes are guaranteed fast turnaround times, making it well-suited for interactive or mixed workloads where some tasks are significantly shorter than others.

#### Limitations:

- SRT suffers from the potential for starvation. Long processes may be repeatedly preempted and pushed back indefinitely if a continuous stream of shorter processes keeps arriving in the ready queue.
- The algorithm incurs higher scheduling overhead than SJF due to frequent context switches triggered at every new arrival, which consumes CPU time and can reduce the practical speed advantage over simpler algorithms.
- Predicting or knowing burst times in advance is generally not feasible in real systems, making SRT difficult to implement outside of simulation or batch environments where execution times are known ahead of time.

### 2.3.4. Test Cases

Table 5. SRT Test Case #1

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 7          |
| P2      | 2            | 4          |
| P3      | 4            | 1          |

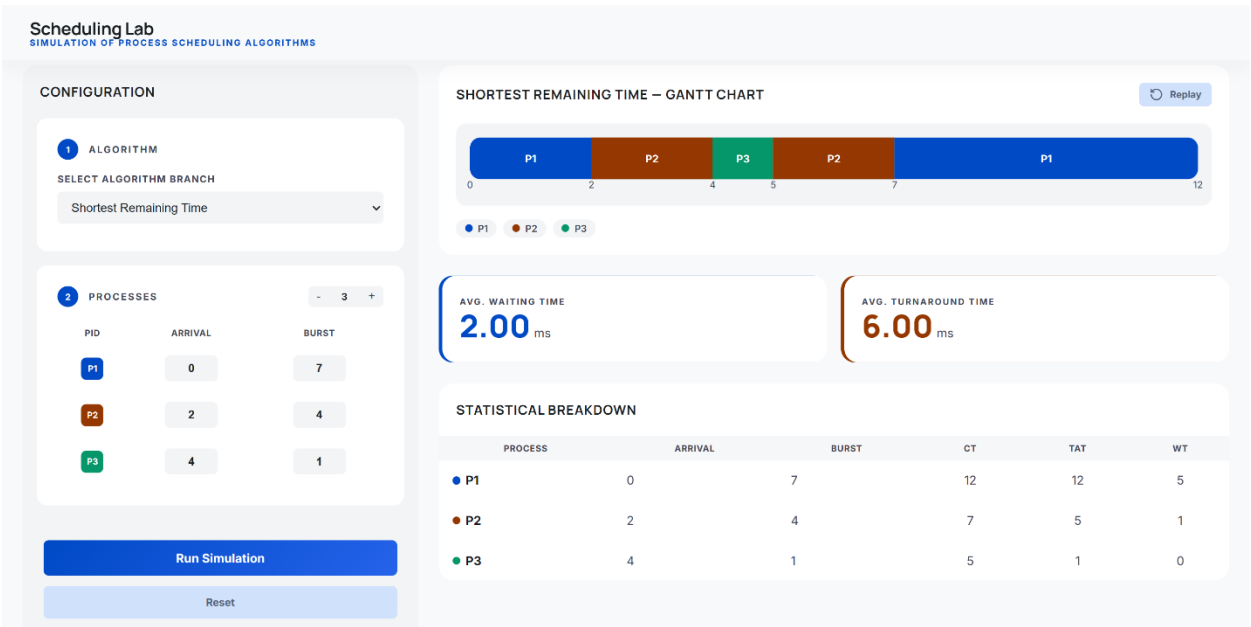


Figure 5. SRT Test Case #1

This test case demonstrates preemption clearly. P1 begins at time 0 but is interrupted at time 2 when P2 arrives with a shorter remaining time. P1 is interrupted again at time 4 when P3 arrives with only 1 unit of burst time. P3 finishes first, then P2 resumes, and P1 completes last.

Table 6. SRT Test Case #2

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 9          |
| P2      | 1            | 3          |
| P3      | 2            | 6          |
| P4      | 3            | 2          |

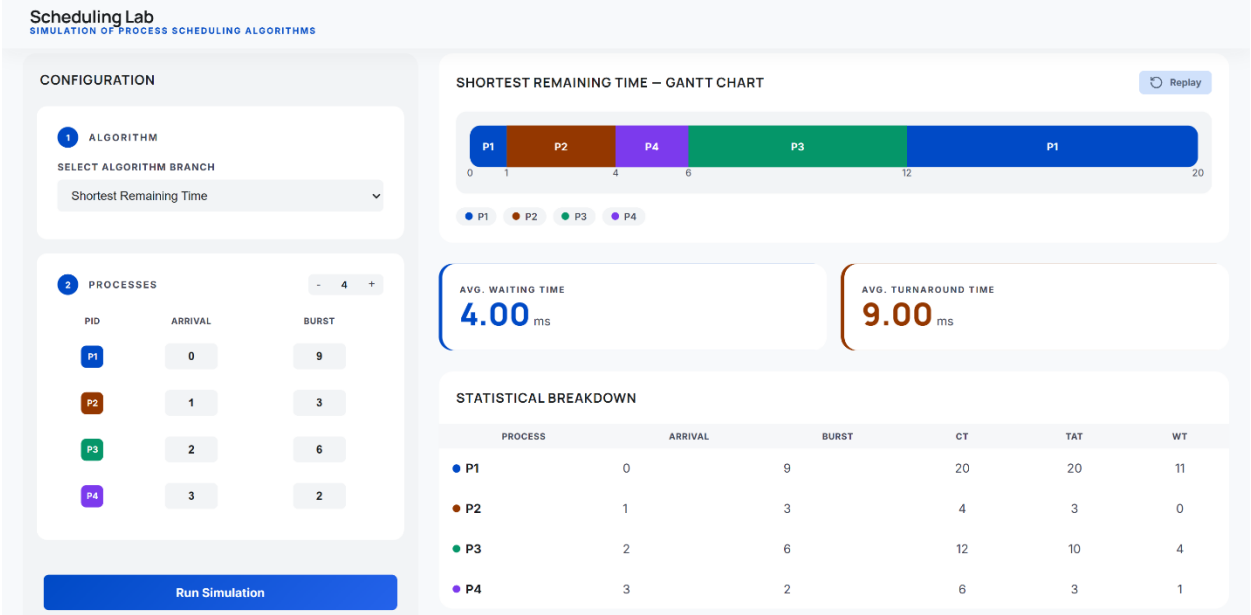


Figure 6. SRT Test Case #2

P1 starts at t=0 but is preempted at t=1 by shorter P2, which runs to t=4 with no wait. P4 (burst 2) runs next and finishes at t=6, followed by P3 at t=12. P1 resumes last and completes at t=20, with a wait time of 11, showing SRT favors short jobs at the expense of longer ones.

## 2.4.Round Robin

### 2.4.1. How It Works

The Round Robin scheduling algorithm is one of the CPU scheduling algorithms in which every process gets a fixed amount of time quantum to execute. In this algorithm, every process gets executed cyclically. This means that processes that have their burst time remaining after the expiration of the time quantum are sent back to the ready state and wait for their next turn to complete execution until it terminates. This processing is done in FIFO order, which suggests that processes are executed on a first-come, first-serve basis (GeeksforGeeks, 2026).

A small unit of time is used in Round Robin, called the Time Quantum (TQ) or Time Slice (TS). The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to one TQ. If a new process arrives, it is added to the tail of the circular queue. The CPU scheduler picks the first process from the queue, sets a timer to interrupt after one TQ, and dispatches the process. After the TQ expires, the CPU preempts the process and the process is added to the tail of the circular queue (Tutorialspoint, n.d).

### 2.4.2. Preemptive or non-preemptive

Round Robin is a preemptive scheduling algorithm. A running process can be interrupted by another process and sent to the ready queue even when it has not completed its entire execution in the CPU. It is considered a preemptive version of the First-Come, First-Served (FCFS) scheduling algorithm.

### 2.4.3. Advantages and Limitations

#### Advantages

- Each process receives a fair share of CPU time.
- Its cyclic scheduling prevents starvation.

#### Limitations

- A very short time quantum increases overhead and reduces CPU efficiency, while a long quantum hurts responsiveness for short jobs.
- Average waiting time is usually high.
- If the time quantum is too large, RR effectively becomes FCFS.

### 2.4.4. Test Cases

Table 7. RR Test Case #1

| Process          | Arrival Time | Burst Time |
|------------------|--------------|------------|
| P1               | 0            | 5          |
| P2               | 1            | 3          |
| P3               | 2            | 4          |
| Time Quantum = 2 |              |            |

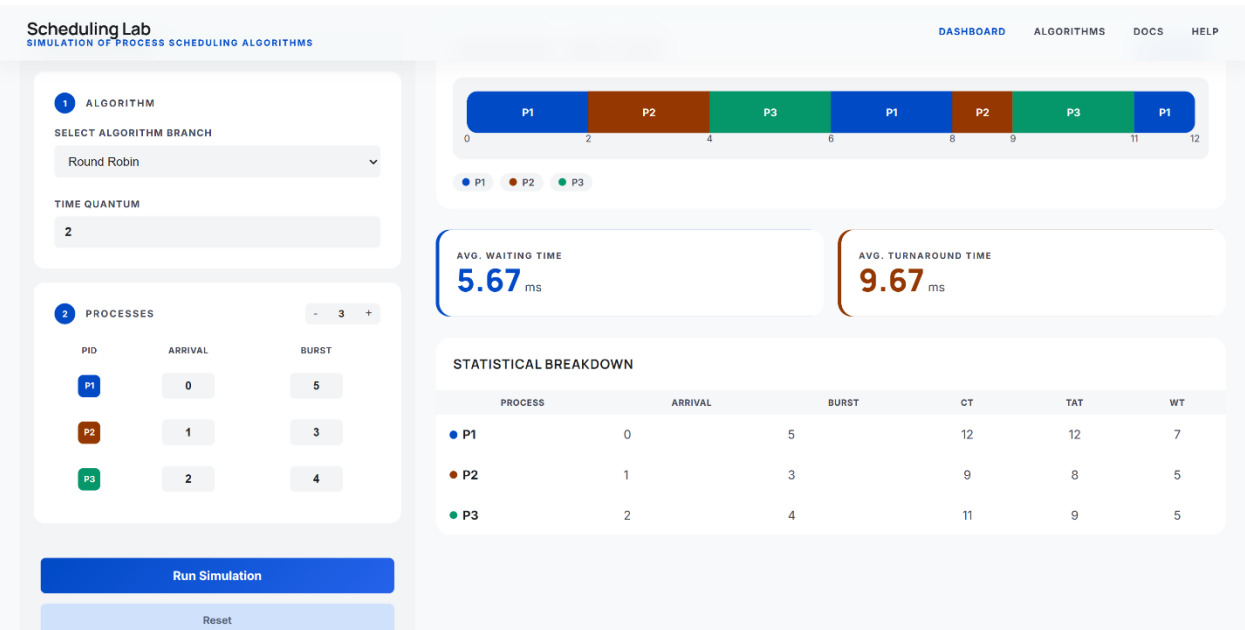


Figure 7. RR Test Case #1

This test case uses a time quantum of 2, which is small relative to the burst times, resulting in multiple rotations across all three processes before any of them completes. It clearly illustrates the cyclic nature of Round Robin and how the ready queue cycles through each process repeatedly.

Table 8. RR Test Case #2

| Process          | Arrival Time | Burst Time |
|------------------|--------------|------------|
| P1               | 0            | 6          |
| P2               | 1            | 4          |
| P3               | 3            | 2          |
| P4               | 5            | 5          |
| Time Quantum = 3 |              |            |

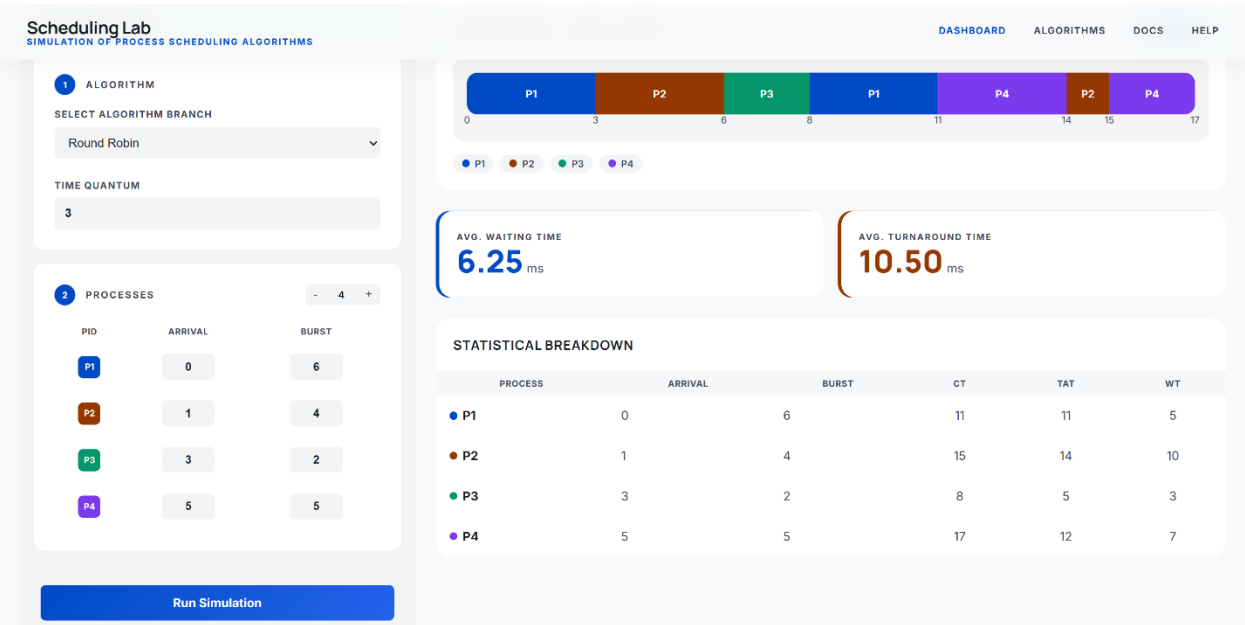


Figure 8. RR Test Case #2

This test case uses a larger time quantum of 3. P3, with only 2 units of burst time, completes within a single quantum without being preempted, demonstrating how processes shorter than the time quantum behave identically to FCFS within their assigned slot.



## 2.5. Priority Scheduling

### 2.5.1. How It Works

Priority scheduling is one of the most common scheduling algorithms used by the operating system to schedule processes based on their priority. Each process is assigned a priority value based on criteria such as memory requirements, time requirements, other resource needs, or the ratio of average I/O to average CPU burst time. The process with the highest priority is selected for execution first. If there are multiple processes sharing the same priority, they are scheduled in the order they arrived, following a First-Come, First-Served approach. It is important to note that priority conventions differ across implementations. In some systems, a higher numerical value means higher priority (e.g., priority 5 is more urgent than priority 1), while in others, a lower numerical value means higher priority (e.g., priority 1 is the most urgent). This case study's simulator supports both conventions, and the priority convention used will be specified clearly in each test case below (GeeksforGeeks, 2025).

Priority scheduling can be implemented in two ways. In non-preemptive priority scheduling, the CPU is allocated to a process based on its priority and will not be released until the process either completes execution or voluntarily enters a waiting state, even if a higher-priority process arrives during that time. Non-preemptive priority scheduling that uses static priorities is typically used in batch processes. In preemptive priority scheduling, if a process arrives that has a higher priority than the currently executing process, the higher-priority process preempts the lower-priority one. The preempted process is moved back to the ready queue, and the new process takes over the CPU. Preemptive priority scheduling that uses dynamic priority is used in most modern operating systems (Marthur, 2024).

### 2.5.2. Preemptive or non-preemptive

Priority Scheduling can be implemented as either preemptive or non-preemptive, and this case study covers both variants. The non-preemptive variant allows a running process to finish before a newly arrived higher-priority process takes over, while the preemptive variant immediately displaces the running process in favor of the higher-priority arrival.

### 2.5.3. Advantages and Limitations

#### Advantages

- Priority-based (PB) scheduling allows processes to be assigned different priority levels based on factors such as importance or urgency.
- Since most systems have some high-priority system processes, priority scheduling finds wide implementation, often in conjunction with other scheduling algorithms (GeeksforGeeks, 2025).

#### Limitations

- If high-priority processes consume most of the CPU time, lower-priority processes may be delayed indefinitely. This condition is known as starvation, where a process never gets scheduled to run.
- Another issue is determining the appropriate priority level to assign to each process.
- In the event of a system crash, all low-priority processes that have not yet been executed may be lost entirely, since they remain in memory indefinitely while higher-priority processes are served first (Brooks, 2024).

2.5.4. Test Cases

Table 9. Priority Test Case #1 Lower Value = Higher Priority & Non-preemptive

| Process | Arrival Time | Burst Time | Priority |
|---------|--------------|------------|----------|
| P1      | 0            | 5          | 3        |
| P2      | 1            | 3          | 1        |
| P3      | 2            | 4          | 2        |

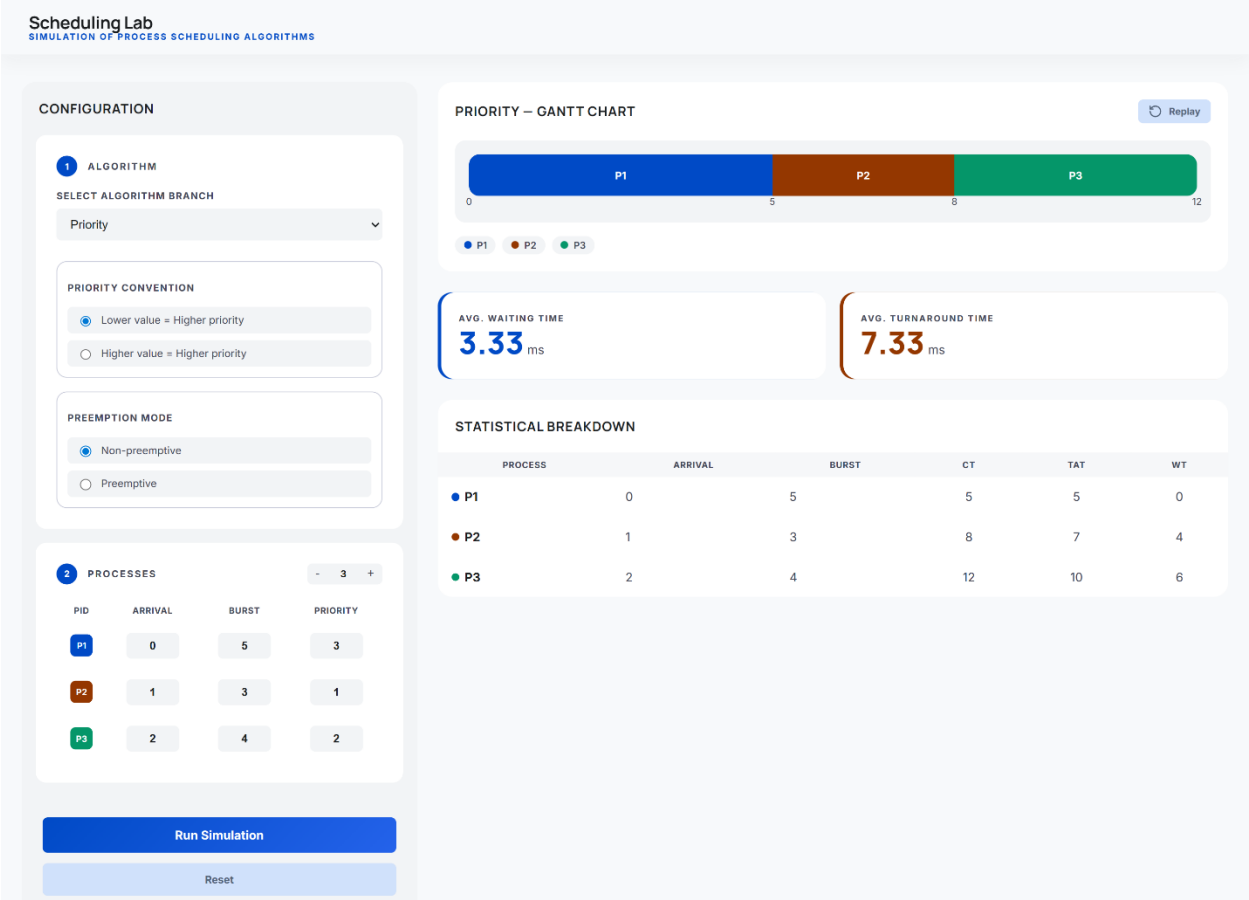


Figure 9. Priority Test Case #1 Lower Value = Higher Priority & Non-preemptive

P1 begins at time 0 since it is the only process available. Once it completes at time 5, both P2 and P3 have already arrived. Since this is non-preemptive, P1 ran to completion without interruption. At time 5, P2 is selected next due to its priority value of 1 being the highest, followed by P3 with priority 2.

Table 10. Priority Test Case #2 Higher Value = Higher Priority & Preemptive

| Process | Arrival Time | Burst Time | Priority |
|---------|--------------|------------|----------|
| P1      | 0            | 6          | 2        |
| P2      | 1            | 4          | 3        |
| P3      | 2            | 5          | 1        |

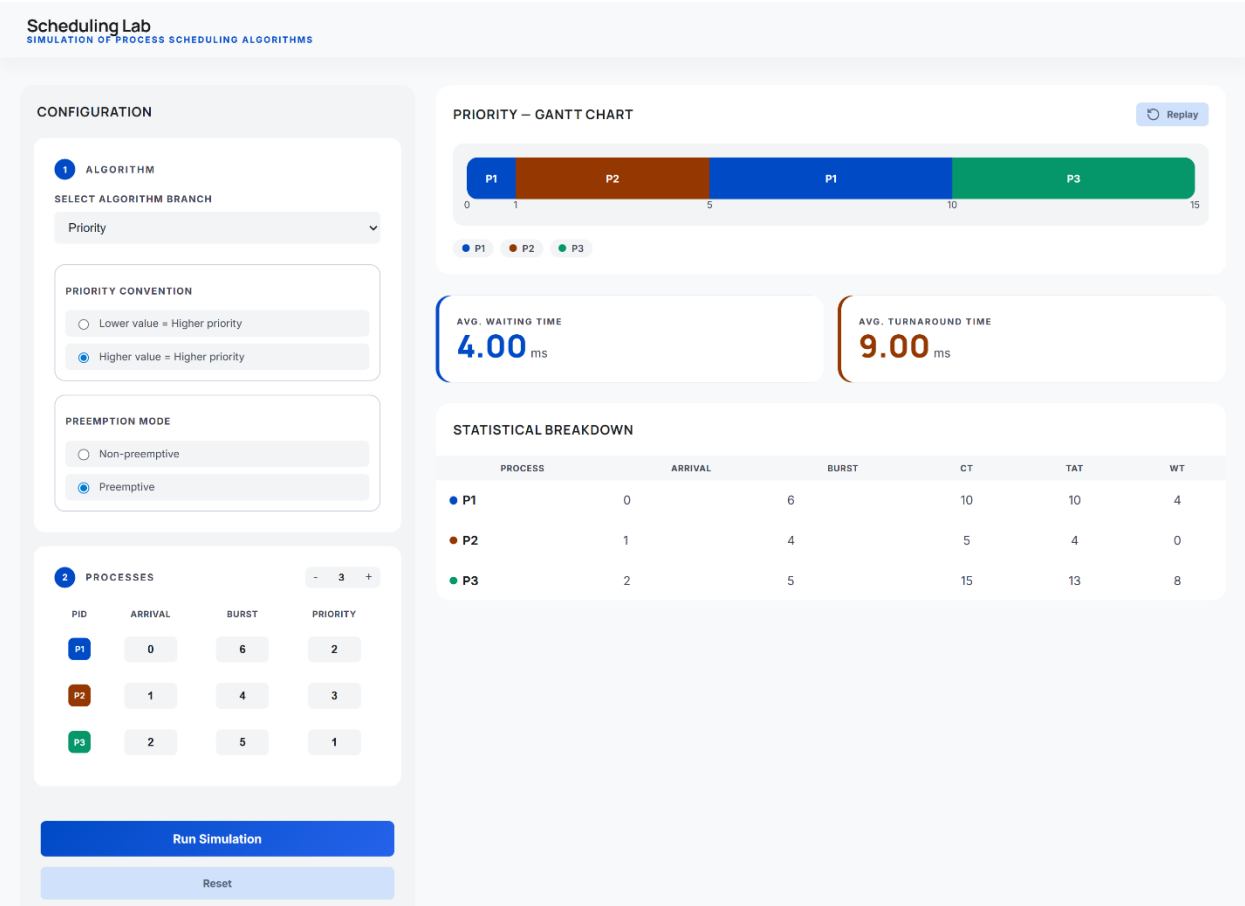


Figure 10. Priority Test Case #2 Higher Value = Higher Priority & Preemptive

This test case shows one preemption using the higher-value priority convention. P1 begins at time 0 but is immediately preempted at time 1 when P2 arrives with a higher priority of 3. At time 2, P3 arrives with priority 1, the lowest of all, so P2 continues uninterrupted. P2 finishes at time 5, after which P1 resumes since its priority 2 beats P3's priority 1. P1 completes at time 10 and P3 runs last, finishing at time 15. P3 suffers the longest wait of 8 units, penalized entirely for having the lowest priority despite arriving early.

## 2.6. Priority Scheduling with Round Robin

### 2.6.1. How It Works

Priority Scheduling with Round Robin is a hybrid CPU scheduling algorithm that combines the priority-based selection of Priority Scheduling with the time-sharing fairness of Round Robin. Processes are executed based on their priority, and processes with specific priorities associated with them on arrival are compared with other processes in the ready queue. When a process arrives, its priority is compared with the priorities of all other processes currently waiting, and the highest-priority process among them is selected to run. However, unlike pure Priority Scheduling where a process runs to completion, each selected process is only allowed to execute for a fixed time quantum before being returned to the ready queue, where the priority comparison occurs again. The operating system assigns a fixed priority to every process, and the scheduler arranges the processes in the ready queue in order of their priority. CPU time is allocated to every process in a round robin fashion according to the given priority, for the given time quantum (GeeksforGeeks, 2025).

The key distinction of this combined approach is how it handles processes that share the same priority level. Processes that have the same priority are served in a Round Robin manner, cycling through them with the assigned time quantum until all processes of that priority group are

completed, before moving on to the next lower priority group. This design directly addresses the most significant weakness of pure priority scheduling. The proposed Priority based Round Robin CPU scheduling algorithm retains the advantage of Round Robin in reducing starvation while also integrating the advantage of priority scheduling, combining the strengths of both algorithms into a single, more balanced strategy (GeeksforGeeks, 2025).

2.6.2. Preemptive or non-preemptive

Priority Scheduling with Round Robin is a preemptive scheduling algorithm. A running process is preempted once its time quantum expires, and it is also preempted if a newly arrived process carries a higher priority, ensuring that high-priority processes are never made to wait behind lower-priority ones beyond a single quantum.

2.6.3. Advantages and Limitations

Advantages

- The algorithm retains the advantage of Round Robin in reducing starvation, ensuring that no process is left waiting indefinitely regardless of its priority level.
- High-priority processes are still given preferential CPU access, making the algorithm suitable for systems where certain tasks are time-critical, while lower-priority processes are still guaranteed eventual execution through the Round Robin cycling mechanism (GeeksforGeeks, 2025).
- The integration of priority and round robin improves performance in terms of average waiting time, average turnaround time, and fair handling of processes across different priority levels compared to using either algorithm independently.

Limitations

- Starvation of lower-priority processes is still possible if many higher-priority processes keep arriving continuously, as the scheduler will always favor the higher-priority group before cycling back to lower-priority ones. Scribd
- The quantum size must be carefully chosen. A larger quantum size leads to longer waiting times for processes of the same priority group, while a smaller quantum size results in increased context-switching overhead. Educative
- The algorithm introduces more complexity in implementation compared to either pure Priority Scheduling or pure Round Robin, as it requires maintaining separate priority-ordered groups within the ready queue while simultaneously tracking remaining burst times for each process across multiple time quantum.

2.6.4. Test Cases

Table 11. Priority + RR Test Case #1 Lower Value = Higher Priority

| Process          | Arrival Time | Burst Time | Priority |
|------------------|--------------|------------|----------|
| P1               | 0            | 3          | 3        |
| P2               | 1            | 6          | 4        |
| P3               | 3            | 1          | 9        |
| P4               | 2            | 2          | 7        |
| P5               | 4            | 4          | 8        |
| Time Quantum = 2 |              |            |          |

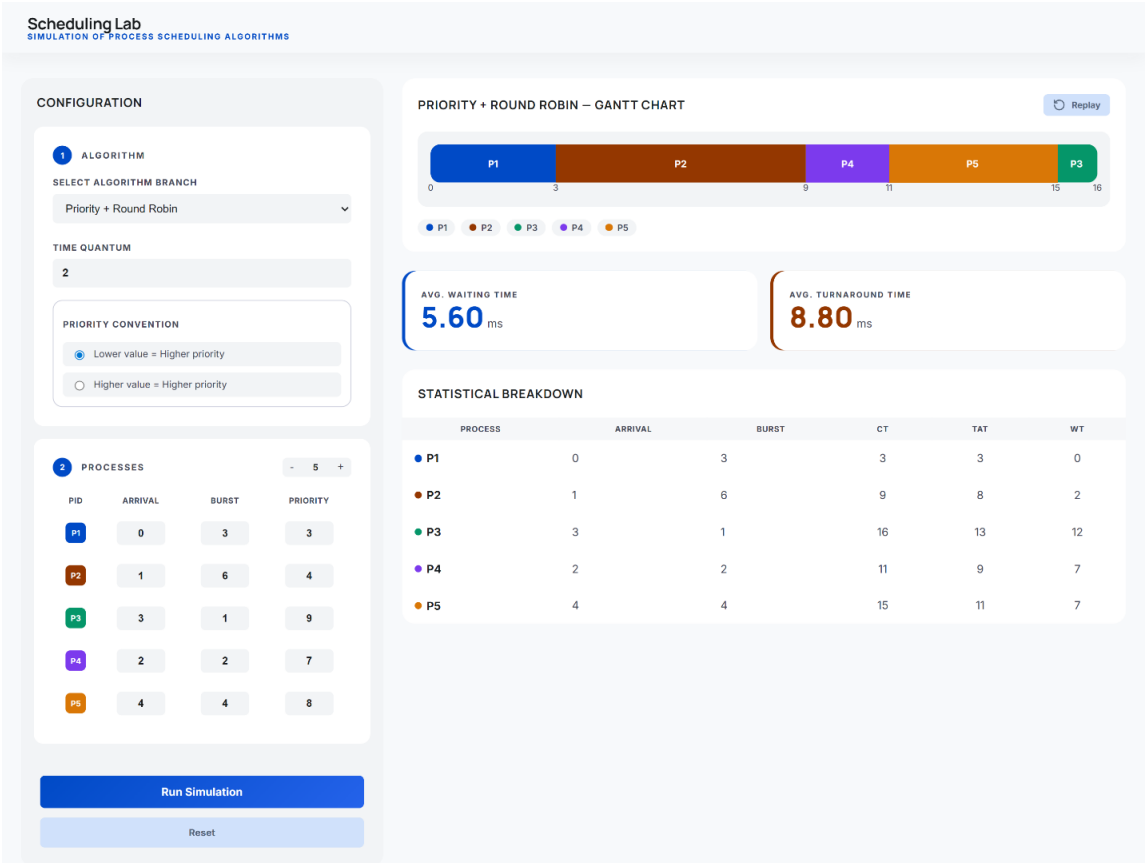


Figure 11. Priority + RR Test Case #1 Lower Value = Higher Priority

P1 has the highest priority (value 3) and runs first. P2 is next with priority 4 and runs for its time quantum of 2 before being cycled. Lower-priority processes P4, P5, and P3 are served in subsequent rounds after higher-priority processes have been handled. This demonstrates how the algorithm prioritizes urgency while still cycling through processes fairly within the same priority level.

Table 12. Priority + RR Test Case #2 Higher Value = Higher Priority

| Process          | Arrival Time | Burst Time | Priority |
|------------------|--------------|------------|----------|
| P1               | 0            | 5          | 1        |
| P2               | 0            | 4          | 1        |
| P3               | 0            | 3          | 2        |
| Time Quantum = 2 |              |            |          |

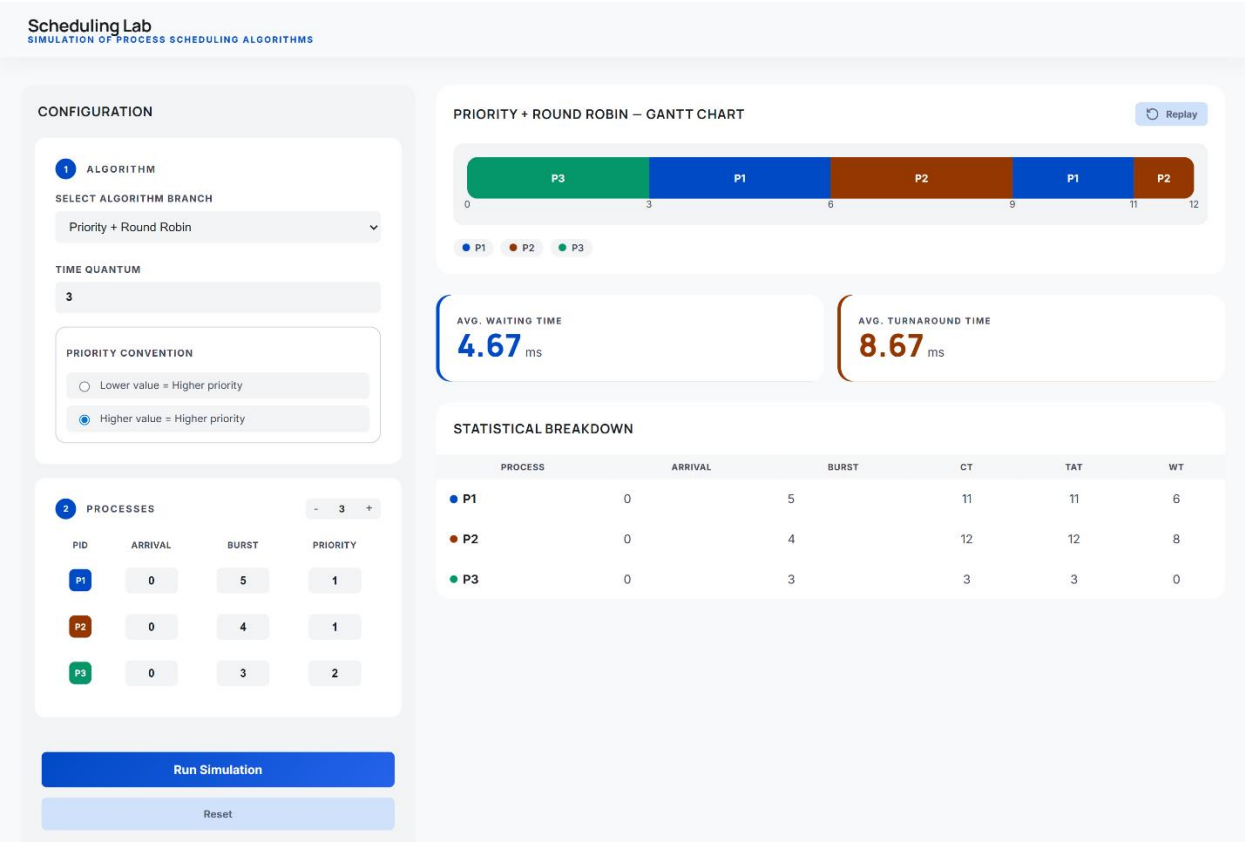


Figure 12. Priority + RR Test Case #2 Higher Value = Higher Priority

Since higher values indicate higher priority, P3 (priority 2) executes first and completes before others. After that, P1 and P2 (priority 1) are scheduled using Round Robin with a time quantum of 2, alternating until both finishes. This shows that higher-priority processes are completed first, while equal-priority processes share the CPU fairly through time slicing.

## 3. Summary

### 3.1. Key Insights

Implementing a web-based CPU scheduling simulator provided a level of understanding that goes far beyond reading algorithm descriptions in a textbook. One of the most significant realizations throughout this project is that each scheduling algorithm is not simply a variation of the same idea, but a distinct design philosophy with its own set of assumptions about what a "good" outcome looks like. FCFS assumes fairness through arrival order. SJF and SRT optimize for throughput at the cost of fairness. Round Robin treats all processes as equals regardless of their urgency. Priority Scheduling acknowledges that not all processes carry the same weight. Priority with Round Robin attempts to find a middle ground between all these competing values.

Another key insight is how sensitive scheduling outcomes are to input values. A small change in arrival time or burst time can drastically shift waiting times and turnaround times, sometimes causing a process that would have finished quickly under one algorithm to suffer significantly under another. This sensitivity became especially apparent when tracing through the SRT and Priority with Round Robin test cases, where the order of preemption events determined the entire execution sequence.

Building the simulator also revealed the hidden complexity behind concepts that initially appeared straightforward. Implementing the Gantt chart rendering, correctly tracking remaining burst times during preemption, and ensuring that the ready queue was updated at every time unit for preemptive algorithms required careful attention to edge cases that no textbook example fully captures.

### 3.2. Performance Differences Observed

Across all six algorithms tested in this case study, clear patterns emerged in terms of fairness, efficiency, and response time.

FCFS consistently produced the highest average waiting times in cases where a long process arrived before shorter ones, a direct consequence of the Convoy Effect. While it is completely fair in the sense that every process eventually gets executed in order of arrival, it is the least efficient algorithm when process burst times vary significantly.

SJF improved on FCFS by always selecting the shortest available job, resulting in lower average waiting times under the same process sets. However, its non-preemptive nature means that a long process already running will not be interrupted even if a very short process arrives moments later. SRT corrects this by preempting at every new arrival, achieving the lowest average waiting times among all algorithms tested, though at the cost of significantly more context switches and the risk of starvation for longer processes.

Round Robin demonstrated the most consistent and predictable behavior in terms of response time. Because every process receives CPU time within each cycle, no process waits excessively for its first turn. However, average turnaround times under Round Robin were generally higher than SRT and SJF, particularly when the time quantum was not well-matched to the process burst times in the test cases.

Priority Scheduling offered a clear advantage in systems where process urgency matters. High-priority processes experienced near-zero waiting times in the preemptive variant, while lower-priority processes suffered as a result. Priority Scheduling with Round Robin moderated this imbalance by cycling through same-priority processes fairly, offering a more balanced distribution of waiting times across the full process set without completely abandoning the priority structure.

### 3.3. Practical Relevance in Modern OS

CPU scheduling is not a theoretical construct confined to academic exercises. It is an active and continuous process happening inside every device running a modern operating system. Windows, Linux, and macOS all implement variations of the algorithms covered in this case study, often as hybrid or multilevel feedback queue approaches that adapt dynamically to the behavior of running processes.

The Linux Completely Fair Scheduler (CFS), for instance, draws from the spirit of Round Robin and Priority Scheduling, assigning virtual runtimes to processes and always selecting the process that has received the least CPU time relative to its weight. Real-time operating systems used in embedded devices, medical equipment, and aerospace systems rely heavily on preemptive priority scheduling to guarantee that critical tasks always meet their deadlines, directly reflecting the Priority Scheduling algorithm explored in this case study.

Understanding how these fundamental algorithms behave under different process loads builds the analytical foundation needed to reason about why modern schedulers are designed the way they are, and what tradeoffs system designers accept when choosing one scheduling strategy over another. The insights from running even a simplified simulation of these six algorithms provide a concrete basis for understanding those tradeoffs at a practical level.

### 3.4. Challenges & How They Were Addressed

Several technical and conceptual challenges were encountered during the development and testing of this simulator.

The most significant challenge was correctly implementing the preemptive algorithms, particularly SRT and Priority Scheduling (Preemptive). Unlike non-preemptive algorithms where execution is straightforward, preemptive algorithms required simulating the passage of time unit by unit, checking at each tick whether a newly arrived process should displace the currently running one. Early versions of the simulator produced incorrect Gantt charts for these algorithms because the ready queue was not being updated at every time unit, only at the moment of completion. This was resolved by restructuring the simulation loop to check for new arrivals and compare remaining burst times at every single time unit throughout execution.

Generating the Gantt chart dynamically in the browser presented a separate challenge. Because the number of blocks in the chart varies depending on the algorithm and input values, the chart had to be rendered programmatically through JavaScript rather than as a static layout. Handling consecutive blocks belonging to the same process, and ensuring that the timeline numbers beneath the chart aligned correctly with each block boundary, required multiple rounds of testing and adjustment across different process inputs and algorithms.

Another challenge involved validation of user inputs. The simulator needed to gracefully handle edge cases such as multiple processes with the same arrival time, processes with equal priority, and missing inputs for priority or time quantum fields when an algorithm that requires them is selected. These cases were addressed by adding input validation checks before the simulation begins, with clear error messages directing the user to correct the input before proceeding.

Finally, verifying the correctness of the computed metrics across all six algorithms was time-consuming but essential. Each test case was manually traced step by step before being entered into the simulator, and the simulator's output was compared against the manual computation to confirm accuracy. Discrepancies led to debugging sessions that ultimately strengthened both the correctness of the algorithm implementations, and a deeper understanding of how each scheduling decision affects the final metrics.



### 3.5. Cross-Algorithm Comparison

The six scheduling algorithms implemented in this case study each reflect a different design priority, and their performance characteristics differ depending on the nature of the process workload.

In terms of waiting time and throughput, SRT consistently produced the lowest average waiting times, followed closely by SJF, as both algorithms favor shorter processes by design. FCFS produced the highest average waiting times in most cases due to the Convoy Effect, where short processes are forced to wait behind longer ones. Round Robin, Priority Scheduling, and Priority with Round Robin fell in between, with their results heavily dependent on time quantum size and priority distribution.

Regarding fairness and starvation, FCFS and Round Robin do not starve any process by design. SJF and SRT are the least fair, as long processes can be repeatedly bypassed or preempted indefinitely. Priority Scheduling introduces deliberate asymmetry that benefits urgent tasks but risks starving lower-priority ones. Priority with Round Robin moderates this by cycling through equal-priority processes fairly, offering a more balanced distribution than pure Priority Scheduling.

For response time, Round Robin provides the most consistent and predictable response among all algorithms, making it well-suited for interactive systems. Preemptive Priority Scheduling delivers fast response for high-priority processes but at the direct expense of lower-priority ones. FCFS offers the worst response time in mixed workloads.

No single algorithm is optimal for all workloads. The choice depends on the specific requirements of the system, whether the priority is throughput, fairness, responsiveness, or simplicity.

## 4. References

- GeeksforGeeks. (2025, July 12). *Advantages and Disadvantages of various CPU scheduling algorithms*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/advantages-and-disadvantages-of-various-cpu-scheduling-algorithms/>
- GeeksforGeeks. (2026, January 6). *Round Robin Scheduling in Operating System*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/round-robin-scheduling-in-operating-system/>
- GeeksforGeeks. (2026, January 7). *Shortest Job First or SJF CPU Scheduling*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/shortest-job-first-or-sjf-cpu-scheduling/>
- Tutorialspoint. (n.d). *Round Robin (RR) Scheduling Algorithm*. Tutorialspoint.Com. [https://www.tutorialspoint.com/operating\\_system/os\\_round\\_robin\\_scheduling\\_algorithm.htm](https://www.tutorialspoint.com/operating_system/os_round_robin_scheduling_algorithm.htm)
- Brooks, N. (2024, August 12). *Priority Scheduling Algorithm: Preemptive, Non-Preemptive EXAMPLE*. Guru99. <https://www.guru99.com/priority-scheduling-program.html>
- Buck, J., & Perugini, S. (2020, November 18). *An Interactive, Graphical CPU Scheduling Simulator for Teaching Operating Systems*. Arxiv. <https://arxiv.org/pdf/1812.05160>
- GeeksforGeeks. (2017, November 3). *Priority CPU Scheduling with different arrival time Set 2*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/priority-cpu-scheduling-with-different-arrival-time-set-2/>
- GeeksforGeeks. (2019, November 7). *Introduction of Shortest Remaining Time First (SRTF) algorithm*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/introduction-of-shortest-remaining-time-first-srtf-algorithm/>
- GeeksforGeeks. (2020, June 8). *Difference between Priority Scheduling and Round Robin (RR) CPU scheduling*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/difference-between-priority-scheduling-and-round-robin-rr-cpu-scheduling/>
- GeeksforGeeks. (2020, October 8). *Priority to Roundrobin scheduling with dynamic time quantum*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/priority-to-round-robin-scheduling-with-dynamic-time-quantum/>
- GeeksforGeeks. (2025, January 13). *Priority Scheduling in Operating System*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/priority-scheduling-in-operating-system/>
- GeeksforGeeks. (2026, April 24). *CPU Scheduling in Operating Systems*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/cpu-scheduling-in-operating-systems/>
- González-Rodríguez, M., Otero-Cerdeira, L., González-Rufino, E., & Rodríguez-Martínez, F. J. (2024). Study and evaluation of CPU scheduling algorithms. *Heliyon*, 10(9), e29959. <https://doi.org/10.1016/j.heliyon.2024.e29959>

- Goyal, S. (2024, February 16). *CPU Scheduling In Operating Systems | All Concepts Explained*. Unstop.Com; Unstop. <https://unstop.com/blog/cpu-scheduling-in-operating-system>
- Mathur, T. (2022, April 28). *Scaler*. Scaler Topics. <https://www.scaler.com/topics/operating-system/cpu-scheduling/>
- Mathur, T. (2022, February 17). *Scaler*. Scaler Topics. <https://www.scaler.com/topics/operating-system/priority-scheduling-algorithm/>
- Peter, A. (2023, January 10). *Baeldung*. Baeldung on Computer Science. <https://www.baeldung.com/cs/cpu-scheduling>
- Saleem, U., & Javed, M. Y. (2026). Simulation of CPU scheduling algorithms. *2000 TENCON Proceedings. Intelligent Systems and Technologies for the New Millennium (Cat. No.00CH37119)*, 2, 562–567. <https://doi.org/10.1109/tencon.2000.888801>
- Shandilya, P. (2024). *CPU Scheduling: Principles and Challenges*. Parthshandilya.Com. <https://www.parthshandilya.com/cpu-scheduling-principles-and-challenges/>
- Tutorialspoint. (2026). *Shortest Job First (SJF) Scheduling*. Tutorialspoint.Com. [https://www.tutorialspoint.com/operating\\_system/os\\_shortest\\_job\\_first\\_scheduling.htm](https://www.tutorialspoint.com/operating_system/os_shortest_job_first_scheduling.htm)
- Wadmare, J., Kolte, D., Bhatia, K., Desai, P., & Wadmare, G. (2024). Virtual Simulations Tool for Operating Systems: Advancing E-Learning in Computing Education. *Journal of Information Technology Education: Innovations in Practice*, 23, 018. <https://doi.org/10.28945/5404>

## 5. Repository Link

<https://github.com/sudo-paoo/cpu-process-scheduling-algorithms>

## 6. Source Code

### *index.html*

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport"
content="width=device-width, initial-
scale=1.0" />
  <title>Process Scheduling</title>
  <!-- Font -->
  <link rel="preconnect"
href="https://fonts.googleapis.com" />
  <link rel="preconnect"
href="https://fonts.gstatic.com" crossorigin
/>
  <link rel="stylesheet"
href="https://fonts.googleapis.com/css2?fam
ily=Inter:wght@400;600;700&family=Manr
ope:wght@600;700;800&display=swap" />
  <!-- Anime.js -->
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/a
nimejs/3.2.1/anime.min.js"></script>
  <!-- Lucide icons -->
  <script
src="https://unpkg.com/lucide@latest"></sc
ript>
  <!-- Main styles -->
  <link rel="stylesheet"
href="assets/css/styles.css" />
</head>
<body>
  <!-- Top bar -->
  <header class="topbar">
    <div class="topbar-brand">
      <span class="brand-title">Scheduling
Lab</span>
      <span class="brand-subtitle">Simulation
of Process Scheduling Algorithms</span>
    </div>
  </header>
  <!-- App content -->
  <main class="app-main">
    <div class="app-shell">
      <!-- Config panel -->
      <aside class="panel config-panel">
        <div class="panel-header">
          <h2 class="panel-
title">Configuration</h2>
        </div>
        <section class="config-section">
          <div class="section-heading">
            <span class="step-badge">1</span>
```

```
      <span class="section-
label">Algorithm</span>
    </div>
    <!-- Algorithm select -->
    <label class="input-label"
for="algorithm-select">Select Algorithm
Branch</label>
    <select id="algorithm-select"
class="algo-select">
      <option value="" selected>Choose
one...</option>
      <option value="FCFS">First Come,
First Served</option>
      <option value="SJF">Shortest Job
First</option>
      <option value="SRT">Shortest
Remaining Time</option>
      <option value="RR">Round
Robin</option>
      <option
value="Priority">Priority</option>
      <option value="PriorityRR">Priority
+ Round Robin</option>
    </select>
    <!-- Quantum input -->
    <div class="quantum-group hidden"
id="quantum-container">
      <label class="input-label" for="time-
quantum">Time Quantum</label>
      <input id="time-quantum"
class="input-field" type="number" min="1"
max="100" value="2" />
    </div>
    <!-- Priority mode -->
    <div class="option-group hidden"
id="priority-mode-container">
      <p class="input-label">Priority
Convention</p>
      <label class="radio-row">
        <input type="radio"
name="priority-mode" value="lower-
higher" checked />
        <span>Lower value = Higher
priority</span>
      </label>
      <label class="radio-row">
        <input type="radio"
name="priority-mode" value="higher-
higher" />
        <span>Higher value = Higher
priority</span>
      </label>
    </div>
    <!-- Preemption mode -->
    <div class="option-group hidden"
id="preemption-container">
```

```

    <p class="input-label">Preemption
Mode</p>
    <label class="radio-row">
        <input type="radio"
name="preemption-mode" value="non-
preemptive" checked />
        <span>Non-preemptive</span>
    </label>
    <label class="radio-row">
        <input type="radio"
name="preemption-mode"
value="preemptive" />
        <span>Preemptive</span>
    </label>
</div>
</section>

<!-- Process table -->
<section class="config-section">
    <div class="section-heading section-
heading-between">
        <div class="section-heading-left">
            <span class="step-
badge">2</span>
            <span class="section-
label">Processes</span>
        </div>

        <div class="counter-control"
role="group">
            <!-- Remove process -->
            <button class="counter-btn"
id="btn-remove-process" type="button"
aria-label="Remove process">-</button>
            <span class="counter-value"
id="process-count-display">3</span>
            <!-- Add process -->
            <button class="counter-btn"
id="btn-add-process" type="button" aria-
label="Add process">+</button>
        </div>

        <div class="process-table-wrap">
            <table class="process-table"
id="process-table">
                <thead>
                    <tr>
                        <th>PID</th>
                        <th class="text-
center">Arrival</th>
                        <th class="text-
center">Burst</th>
                        <th class="text-center priority-
header hidden">Priority</th>
                    </tr>
                </thead>

```

```

        <!-- Dynamic rows -->
        <tbody id="process-
tbody"></tbody>
    </table>
</div>
</section>

<div class="panel-actions">
    <!-- Run button -->
    <button class="btn btn-primary btn-
full" id="btn-run" type="button">Run
Simulation</button>
    <!-- Reset button -->
    <button class="btn btn-secondary btn-
full" id="btn-reset"
type="button">Reset</button>
</div>
</aside>
<!-- Output panel -->
<section class="content-column">
    <div class="panel simulation-empty"
id="simulation-empty">
        <h3 class="simulation-title">No
simulation yet</h3>
        <p class="simulation-text">Configure
input, then run from an algorithm branch to
render the Gantt chart and metrics.</p>
    </div>
    <!-- Gantt panel -->
    <div class="panel gantt-panel hidden"
id="gantt-panel">
        <div class="panel-header gantt-panel-
header">
            <div class="gantt-header-left">
                <h3 class="panel-title" id="gantt-
algo-label">Gantt Chart</h3>
            </div>
            <button class="btn btn-secondary
gantt-replay-btn" type="button" id="btn-
replay">
                <i data-lucide="rotate-ccw"
class="icon-small" aria-hidden="true"></i>
                Replay
            </button>
        </div>
        <div class="gantt-body">
            <div class="gantt-canvas-wrap">
                <!-- Gantt bars -->
                <div id="gantt-container"
class="gantt"></div>
                <!-- Time markers -->
                <div id="gantt-time" class="gantt-
time"></div>
            </div>
            <!-- Legend -->
            <div class="gantt-legend" id="gantt-
legend"></div>

```

```

    </div>
  </div>
  <div class="metrics-grid">
    <article class="panel metric-card
metric-card-primary">
      <p class="metric-label">Avg.
Waiting Time</p>
      <h4 class="metric-value metric-
value-primary"><span id="avg-wt-
display">-</span><span class="metric-
unit"> ms</span></h4>
    </article>
    <article class="panel metric-card
metric-card-tertiary">
      <p class="metric-label">Avg.
Turnaround Time</p>
      <h4 class="metric-value metric-
value-tertiary"><span id="avg-tat-
display">-</span><span class="metric-
unit"> ms</span></h4>
    </article>
  </div>
  <div class="panel results-panel">
    <div class="panel-header">
      <h3 class="panel-title">Statistical
Breakdown</h3>
    </div>
    <div class="results-table-wrap">
      <table class="results-table">
        <thead>
          <tr>
            <th>Process</th>
            <th class="text-
center">Arrival</th>
            <th class="text-
center">Burst</th>
            <th class="text-center">CT</th>
            <th class="text-
center">TAT</th>
            <th class="text-center">WT</th>
          </tr>
        </thead>
        <!-- Result rows -->
        <tbody id="results-tbody">
          <tr>
            <td>-</td>
            <td class="text-center">-</td>
            <td class="text-center">-</td>
            <td class="text-center">-</td>
            <td class="text-center">-</td>
          </tr>
        </tbody>
      </table>
    </div>
  </div>
</section>

```

```

    </div>
  </main>
  <footer class="site-footer">
    <p class="footer-text">
      View this project on
      <a class="footer-link"
href="https://github.com/sudo-paoo/cpu-
process-scheduling-algorithms"
target="_blank" rel="noopener
noreferrer">GitHub</a>
    </p>
  </footer>
  <!-- Snackbar -->
  <div id="app-snackbar" class="snackbar"
role="status" aria-live="polite" aria-
atomic="true"></div>
  <!-- Algorithms -->
  <script
src="assets/js/algorithms/fcfs.js"></script>
  <script
src="assets/js/algorithms/sjf.js"></script>
  <script
src="assets/js/algorithms/srt.js"></script>
  <script
src="assets/js/algorithms/rr.js"></script>
  <script
src="assets/js/algorithms/priority.js"></scrip
t>
  <script src="assets/js/algorithms/priority-
rr.js"></script>
  <!-- App script -->
  <script src="assets/js/main.js"
defer></script>
</body>
</html>

/assets/css/styles.css
:root {
  --tertiary: #943700;
  --surface-variant: #e0e3e5;
  --surface-container-low: #f2f4f6;
  --on-surface: #191c1e;
  --primary: #004ac6;
  --inverse-on-surface: #eff1f3;
  --surface-container-lowest: #ffffff;
  --surface-container: #ecef0;
  --on-primary: #ffffff;
  --surface: #f7f9fb;
  --inverse-surface: #2d3133;
  --secondary-container: #d0e1fb;
  --on-surface-variant: #434655;
  --outline-variant: #c3c6d7;
  --outline: #737686;
  --on-secondary-container: #54647a;
  --surface-bright: #f7f9fb;
  --primary-container: #2563eb;

```

```

--font-headline: "Manrope", sans-serif;
--font-body: "Inter", sans-serif;

--radius-md: 0.375rem;
--radius-xl: 0.75rem;
--radius-2xl: 1rem;

--space-1: 0.25rem;
--space-2: 0.5rem;
--space-3: 0.75rem;
--space-4: 1rem;
--space-5: 1.25rem;
--space-6: 1.5rem;
--space-8: 2rem;
}
* {
    box-sizing: border-box;
}
html,
body {
    margin: 0;
    padding: 0;
}
/* Page base */
body {
    font-family: var(--font-body);
    background: var(--surface);
    color: var(--on-surface);
}
.hidden {
    /* Utility to toggle visibility from JS. */
    display: none !important;
}
.text-center {
    text-align: center;
}
/* Top bar */
.topbar {
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    z-index: 50;
    height: 4rem;
    padding: 0 var(--space-8);
    background: rgba(247, 249, 251, 0.88);
    backdrop-filter: blur(12px);
    display: flex;
    align-items: center;
    justify-content: flex-start;
    gap: var(--space-6);
    box-shadow: 0 8px 20px rgba(25, 28, 30, 0.04);
}

```

```

.topbar-brand {
    display: flex;
    flex-direction: column;
}
.brand-title {
    font-family: var(--font-headline);
    font-size: 1.25rem;
    font-weight: 700;
    line-height: 1.2;
    letter-spacing: -0.02em;
}
.brand-subtitle {
    margin-top: -0.1rem;
    font-size: 0.625rem;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.15em;
    color: var(--primary);
}
.app-main {
    padding: 6rem 1.5rem 0rem;
}
/* App layout */
.app-shell {
    max-width: 1500px;
    margin: 0 auto;
    display: grid;
    grid-template-columns: 1fr;
    gap: var(--space-6);
    align-items: start;
}
.panel {
    background: var(--surface-container-lowest);
    border-radius: var(--radius-2xl);
}
/* Config panel */
.config-panel {
    background: var(--surface-container-low);
    display: flex;
    flex-direction: column;
    overflow: hidden;
}
.panel-header {
    padding: var(--space-5);
    display: flex;
    align-items: center;
    gap: var(--space-2);
}
.panel-title {
    margin: 0;
    font-family: var(--font-headline);
    font-size: 0.95rem;
    font-weight: 700;
    letter-spacing: 0.05em;
    text-transform: uppercase;
}

```



```

        color: var(--on-surface);
    }
    .config-section {
        margin: 0 var(--space-4) var(--space-4);
        padding: var(--space-6);
        border-radius: var(--radius-xl);
        background: var(--surface-container-lowest);
    }
    .section-heading {
        display: flex;
        align-items: center;
        gap: var(--space-3);
        margin-bottom: var(--space-4);
    }
    .section-heading-between {
        justify-content: space-between;
    }
    .section-heading-left {
        display: flex;
        align-items: center;
        gap: var(--space-3);
    }
    .step-badge {
        width: 1.5rem;
        height: 1.5rem;
        border-radius: 999px;
        display: inline-flex;
        align-items: center;
        justify-content: center;
        font-size: 0.625rem;
        font-weight: 700;
        background: var(--primary);
        color: var(--on-primary);
    }
    .section-label {
        font-size: 0.6875rem;
        font-weight: 700;
        text-transform: uppercase;
        letter-spacing: 0.14em;
        color: var(--on-surface-variant);
    }
    .input-label {
        display: block;
        margin-bottom: var(--space-2);
        font-size: 0.6875rem;
        font-weight: 700;
        text-transform: uppercase;
        letter-spacing: 0.08em;
        color: var(--on-surface-variant);
    }
    /* Form controls */
    .algo-select,
    .input-field,
    .process-input {
        width: 100%;

```

```

        background: var(--surface-container-low);
        border: 0;
        border-radius: var(--radius-md);
        padding: 0.6rem 0.75rem;
        font-size: 0.875rem;
        font-weight: 600;
        color: var(--on-surface);
        outline: 0;
        transition: background-color 0.2s ease, box-shadow 0.2s ease;
    }
    .algo-select {
        font-weight: 500;
        margin-bottom: var(--space-2);
    }
    .option-group {
        margin-top: var(--space-4);
        padding: var(--space-3);
        border-radius: var(--radius-xl);
        border: 1px solid var(--outline-variant);
        background: var(--surface-container-lowest);
        display: grid;
        gap: var(--space-2);
    }
    .radio-row {
        display: flex;
        align-items: center;
        gap: 0.5rem;
        font-size: 0.75rem;
        color: var(--on-surface-variant);
        padding: 0.45rem 0.6rem;
        border-radius: var(--radius-md);
        background: var(--surface-container-low);
    }
    .quantum-group {
        margin-top: var(--space-4);
    }

    /* Focus ring */
    .algo-select:focus,
    .input-field:focus,
    .process-input:focus,
    .counter-btn:focus-visible,
    .btn:focus-visible {
        /* Centralized focus style for accessible keyboard navigation. */
        box-shadow: 0 0 2px rgba(0, 74, 198, 0.35);
        outline: none;
    }
    .counter-control {
        display: inline-flex;
        align-items: center;

```

```

        border-radius: var(--radius-md);
        overflow: hidden;
        background: var(--surface-container-
low);
    }
    .counter-btn {
        border: 0;
        background: transparent;
        color: var(--on-surface-variant);
        padding: 0.25rem 0.6rem;
        cursor: pointer;
        transition: background-color 0.2s
ease;
    }
    .counter-btn:hover {
        background: var(--surface-container-
lowest);
    }
    .counter-value {
        min-width: 2.25rem;
        text-align: center;
        font-size: 0.6875rem;
        font-weight: 700;
        color: var(--on-surface);
    }
    .process-table-wrap,
    .results-table-wrap {
        overflow-x: auto;
    }

    .process-table,
    .results-table {
        width: 100%;
        border-collapse: collapse;
    }
    .process-table th,
    .results-table th {
        padding: 0.5rem 0.75rem;
        text-align: center;
        font-size: 0.625rem;
        font-weight: 700;
        letter-spacing: 0.05em;
        text-transform: uppercase;
        color: var(--on-surface-variant);
    }
    .process-row:hover,
    .results-table tbody tr:hover {
        background: var(--surface-bright);
    }
    .process-cell {
        padding: 0.75rem 0.5rem;
        text-align: center;
    }
    /* PID chip */
    .pid-pill {
        width: 1.6rem;
        height: 1.6rem;

        display: inline-flex;
        align-items: center;
        justify-content: center;
        border-radius: 0.375rem;
        font-size: 0.625rem;
        font-weight: 700;
        color: #ffffff;
        background: var(--pid-color, var(--
surface-variant));
    }
    .process-input {
        width: 3.8rem;
        margin: 0 auto;
        display: block;
        text-align: center;
        font-size: 0.8125rem;
        padding: 0.45rem 0.55rem;
    }

    .process-input.is-invalid,
    .input-field.is-invalid {
        box-shadow: 0 0 0 2px rgba(186, 26,
26, 0.45);
    }

    .panel-actions {
        margin-top: auto;
        padding: var(--space-6);
        display: grid;
        gap: var(--space-3);
    }

    .btn {
        border: 0;
        border-radius: var(--radius-md);
        font-family: var(--font-body);
        font-weight: 700;
        cursor: pointer;
        display: inline-flex;
        align-items: center;
        justify-content: center;
        gap: var(--space-2);
        transition: transform 0.2s ease, filter
0.2s ease;
    }

    .btn:hover {
        filter: brightness(1.03);
    }

    .btn:active {
        transform: scale(0.98);
    }

    .btn-primary {
        color: var(--on-primary);

```

```

        background: linear-gradient(135deg,
var(--primary), var(--primary-container));
        padding: 0.75rem 1.25rem;
        font-size: 0.875rem;
    }

    .btn-secondary {
        color: var(--on-secondary-container);
        background: var(--secondary-
container);
        padding: 0.7rem 1rem;
        font-size: 0.75rem;
    }

    .btn-full {
        width: 100%;
    }

    /* Snackbar */
    .snackbar {
        /* Live region anchored to the
viewport for validation feedback. */
        position: fixed;
        left: 50%;
        bottom: 1.25rem;
        transform: translate(-50%, 18px);
        min-width: 220px;
        max-width: min(90vw, 420px);
        padding: 0.7rem 0.9rem;
        border-radius: 0.7rem;
        background: var(--inverse-surface);
        color: var(--inverse-on-surface);
        font-size: 0.75rem;
        font-weight: 600;
        opacity: 0;
        pointer-events: none;
        transition: opacity 0.22s ease,
transform 0.22s ease;
        z-index: 120;
    }

    .snackbar.show {
        opacity: 1;
        transform: translate(-50%, 0);
    }

    .snackbar.snackbar-error {
        background: #472527;
        color: #ffdad6;
    }

    .snackbar.snackbar-info {
        background: #1f3a72;
        color: #dbe1ff;
    }

    .content-column {
        display: grid;
        gap: var(--space-6);
    }

    .simulation-empty {
        min-height: 220px;
        display: flex;
        flex-direction: column;
        align-items: center;
        justify-content: center;
        text-align: center;
        padding: var(--space-8);
    }

    .simulation-title {
        margin: 0;
        font-family: var(--font-headline);
        font-size: 1.125rem;
        font-weight: 700;
    }

    .simulation-text {
        margin: var(--space-2) auto 0;
        max-width: 34rem;
        font-size: 0.875rem;
        line-height: 1.6;
        color: var(--on-surface-variant);
    }

    /* Gantt panel */
    .gantt-panel {
        overflow: hidden;
    }

    .gantt-panel-header {
        justify-content: space-between;
    }

    .gantt-header-left {
        display: flex;
        align-items: center;
        gap: var(--space-2);
    }

    .gantt-replay-btn {
        padding: 0.35rem 0.75rem;
        font-size: 0.6875rem;
        flex-shrink: 0;
    }

    .icon-small {
        width: 1rem;
        height: 1rem;
        stroke-width: 2;
    }

    .gantt-body {
        padding: 0 var(--space-5) var(--
space-5);
    }

    .gantt-canvas-wrap {
        /* Allow horizontal scrolling for
wide schedules. */
        background: var(--surface-container-
low);
        border-radius: var(--radius-xl);
    }

```

```

        padding: var(--space-4) var(--space-
4) var(--space-3);
        overflow-x: auto;
    }
    /* Gantt track */
    .gantt {
        display: flex;
        height: 48px;
        border-radius: var(--radius-xl);
        overflow: hidden;
        background: var(--surface-
container);
    }
    /* Gantt segment */
    .block {
        /* Each block represents a scheduled
slice in the timeline. */
        display: flex;
        align-items: center;
        justify-content: center;
        font-weight: 700;
        font-size: 0.75rem;
        color: #ffffff;
        white-space: nowrap;
        background: var(--pid-color, var(--
outline-variant));
        transition: transform 0.2s ease;
    }
    .block:hover {
        transform: scale(1.02);
        z-index: 2;
    }
    .gantt-time {
        position: relative;
        height: 18px;
        font-size: 0.6875rem;
        color: var(--on-surface-variant);
    }
    .gantt-legend {
        display: flex;
        flex-wrap: wrap;
        gap: 0.5rem;
        margin-top: var(--space-4);
    }
    /* Legend item */
    .legend-item {
        display: flex;
        align-items: center;
        gap: 0.375rem;
        font-size: 0.6875rem;
        font-weight: 600;
        color: var(--on-surface-variant);
        background: var(--surface-container-
low);
        padding: 0.25rem 0.625rem;
        border-radius: 999px;
    }

    .legend-dot {
        width: 9px;
        height: 9px;
        border-radius: 50%;
        flex-shrink: 0;
        background: var(--pid-color, var(--
outline-variant));
    }
    .metrics-grid {
        display: grid;
        grid-template-columns: 1fr;
        gap: var(--space-4);
    }
    /* Metric card */
    .metric-card {
        padding: var(--space-6);
    }
    .metric-card-primary {
        box-shadow: inset 4px 0 0 var(--
primary);
    }
    .metric-card-tertiary {
        box-shadow: inset 4px 0 0 var(--
tertiary);
    }
    .metric-label {
        margin: 0 0 var(--space-1);
        font-size: 0.625rem;
        font-weight: 700;
        text-transform: uppercase;
        letter-spacing: 0.14em;
        color: var(--on-surface-variant);
    }
    .metric-value {
        margin: 0;
        font-family: var(--font-headline);
        font-size: 2.25rem;
        font-weight: 800;
        line-height: 1;
    }
    .metric-value-primary {
        color: var(--primary);
    }
    .metric-value-tertiary {
        color: var(--tertiary);
    }
    .metric-unit {
        font-family: var(--font-body);
        font-size: 0.875rem;
        font-weight: 400;
        color: var(--on-surface-variant);
    }
    .results-panel {
        overflow: hidden;
    }
    .results-table thead {

```

```

        background: rgba(242, 244, 246,
0.7);
    }
.results-table td {
    padding: 0.9rem 1rem;
    font-size: 0.875rem;
    color: var(--on-surface-variant);
}

.results-table td:first-child {
    color: var(--on-surface);
    font-weight: 700;
}
.site-footer {
    padding: 1.5rem 0.75rem;
    text-align: center;
}
.footer-text {
    margin: 0;
    font-size: 0.8125rem;
    color: var(--on-surface-variant);
}
.footer-link {
    color: var(--primary);
    font-weight: 700;
    text-decoration: none;
}
.footer-link:hover {
    text-decoration: underline;
}
/* PID badge */
.pid-colored-pill {
    display: inline-flex;
    align-items: center;
    gap: 0.375rem;
    font-weight: 700;
    font-size: 0.875rem;
    color: var(--on-surface);
}
.pid-color-dot {
    width: 9px;
    height: 9px;
    border-radius: 50%;
    flex-shrink: 0;
    background: var(--pid-color, var(--
outline-variant));
}
@keyframes spin-once {
    /* Replay icon spin tied to the replay
button state. */
    from { transform: rotate(0deg); }
    to { transform: rotate(360deg); }
}
.btn-replaying .icon-small {
    animation: spin-once 0.5s ease-out;
}
/* PID color map */

```

```

.pid-color-1 { --pid-color: #004ac6; }
.pid-color-2 { --pid-color: #943700; }
.pid-color-3 { --pid-color: #059669; }
.pid-color-4 { --pid-color: #7c3aed; }
.pid-color-5 { --pid-color: #d97706; }
.pid-color-6 { --pid-color: #0891b2; }
.pid-color-7 { --pid-color: #dc2626; }
.pid-color-8 { --pid-color: #db2777; }
.pid-color-9 { --pid-color: #65a30d; }
.pid-color-10 { --pid-color: #0d9488; }
.pid-color-idle { --pid-color: #c3c6d7; }
/* Desktop breakpoint */
@media (min-width: 900px) {
    /* Two-column layout on larger
screens. */
    .app-shell {
        grid-template-columns:
minmax(320px, 1fr) minmax(0, 2fr);
    }

    .metrics-grid {
        grid-template-columns:
repeat(2, minmax(0, 1fr));
    }
}
@media (max-width: 960px) {
    .topbar {
        padding: 0 var(--space-4);
    }
}
@media (max-width: 640px) {
    /* Compact spacing for narrow
viewports. */
    .app-main {
        padding: 5.5rem 1rem 2rem;
    }
    .config-section,
    .panel-actions,
    .simulation-empty,
    .metric-card {
        padding: var(--space-5);
    }
    .results-table td {
        padding: 0.85rem 1rem;
    }
    .site-footer {
        padding: 0 1rem 0.5rem;
    }
}
/assets/js/main.js
const MIN_PROCESSES = 3;
const MAX_PROCESSES = 10;
const MAX_INPUT_VALUE = 100;
const ALGORITHM_NAMES = {
    FCFS: 'First Come, First Served',
    SJF: 'Shortest Job First',

```

```

SRT: 'Shortest Remaining Time',
RR: 'Round Robin',
Priority: 'Priority',
PriorityRR: 'Priority + Round Robin',
};
let currentAlgorithm = "";
let processCount = 0;
let snackbarHideTimer = null;
let lastResult = null;
const tableBody =
document.getElementById('process-tbody');
const algorithmSelect =
document.getElementById('algorithm-
select');
const processCountDisplay =
document.getElementById('process-count-
display');
const quantumContainer =
document.getElementById('quantum-
container');
const priorityModeContainer =
document.getElementById('priority-mode-
container');
const preemptionContainer =
document.getElementById('preemption-
container');
const snackbarEl =
document.getElementById('app-snackbar');
// UI bindings
document.getElementById('btn-add-
process')?.addEventListener('click',
addProcess);
document.getElementById('btn-remove-
process')?.addEventListener('click',
removeLastProcess);
document.getElementById('btn-
run')?.addEventListener('click',
runSimulation);
document.getElementById('btn-
reset')?.addEventListener('click', resetAll);
document.getElementById('btn-
replay')?.addEventListener('click',
replayAnimation);
algorithmSelect?.addEventListener('change',
event => setAlgorithm(event.target.value));
initTemplate();
function initTemplate() {
  if (window.lucide?.createIcons) {
    window.lucide.createIcons();

```

```

}
const quantumInput =
document.getElementById('time-quantum');
if (quantumInput) {
  applyInputLimiter(quantumInput, 1);
}
// Default rows
for (let i = 0; i < MIN_PROCESSES; i +=
1) {
  addProcess();
}
setAlgorithm("");
}
function setAlgorithm(algo) {
  // Selected algorithm
  currentAlgorithm = algo;
  if (algorithmSelect &&
algorithmSelect.value !== algo) {
    algorithmSelect.value = algo;
  }

  const showQuantum = algo === 'RR' || algo
=== 'PriorityRR';
  const showPriority = algo === 'Priority' ||
algo === 'PriorityRR';
  const showPreemption = algo ===
'Priority';
  // Conditional controls
  quantumContainer?.classList.toggle('hidden'
, !showQuantum);
  priorityModeContainer?.classList.toggle('hid
den', !showPriority);
  preemptionContainer?.classList.toggle('hidd
en', !showPreemption);
  document.querySelectorAll('.priority-
header').forEach(el =>
el.classList.toggle('hidden', !showPriority));
  document.querySelectorAll('.priority-
cell').forEach(el =>
el.classList.toggle('hidden', !showPriority));
}
function addProcess() {
  if (!tableBody || !processCountDisplay)
return;
  if (processCount >= MAX_PROCESSES)
{
    showNotice(`Maximum
${MAX_PROCESSES} processes
allowed.`);

```

```

    return;
  }
  processCount += 1;
  processCountDisplay.textContent =
String(processCount);
  // New process row
  const pid = `P${processCount}`;
  const row = document.createElement('tr');
  row.className = 'process-row';
  row.id = `row-p${processCount}`;

  const pidCell =
document.createElement('td');
  pidCell.className = 'process-cell';
  const pill =
document.createElement('span');
  pill.className = `pid-pill
${getPidColorClass(pid)}`;
  pill.textContent = pid;
  pidCell.appendChild(pill);
  const arrivalCell =
document.createElement('td');
  arrivalCell.className = 'process-cell';
  const arrivalInput =
buildNumberInput('arrival-input', 0, 0);
  arrivalCell.appendChild(arrivalInput);
  const burstCell =
document.createElement('td');
  burstCell.className = 'process-cell';
  const burstInput =
buildNumberInput('burst-input',
processCount * 2, 1);
  burstCell.appendChild(burstInput);
  const priorityCell =
document.createElement('td');
  const showPriority = currentAlgorithm
=== 'Priority' || currentAlgorithm ===
'PriorityRR';
  priorityCell.className = `process-cell
priority-cell ${showPriority ? '' :
'hidden'}`.trim();
  const priorityInput =
buildNumberInput('priority-input',
processCount, 0);
  priorityCell.appendChild(priorityInput);
  row.appendChild(pidCell);
  row.appendChild(arrivalCell);
  row.appendChild(burstCell);
  row.appendChild(priorityCell);

```

```

    tableBody.appendChild(row);
  }

function removeLastProcess() {
  if (!processCountDisplay || processCount
<= MIN_PROCESSES) {
    showNotice(`Minimum
${MIN_PROCESSES} processes
required.`);
    return;
  }

  const lastRow =
document.getElementById(`row-
p${processCount}`);
  if (lastRow) {
    lastRow.remove();
    processCount -= 1;
    processCountDisplay.textContent =
String(processCount);
  }
}

function buildNumberInput(inputClass,
value, min) {
  const input =
document.createElement('input');
  input.type = 'number';
  input.className = `process-input
${inputClass}`;
  input.value = String(value);
  applyInputLimiter(input, min);
  return input;
}

function applyInputLimiter(input, min) {
  input.min = String(min);
  input.max =
String(MAX_INPUT_VALUE);

  const clampValue = () => {
    // Guard against invalid or out-of-range
    inputs on typing and blur.
    const parsed =
Number.parseInt(input.value, 10);
    if (Number.isNaN(parsed)) return;
    if (parsed > MAX_INPUT_VALUE) {
      showNotice(`Value cannot go past
${MAX_INPUT_VALUE}.`);
    }
  }
}

```

```

    input.value =
String(MAX_INPUT_VALUE);
    return;
  }
  if (parsed < min) {
    input.value = String(min);
  }
};

input.addEventListener('input',
clampValue);
input.addEventListener('change',
clampValue);
}

function readProcessInputs() {
  // Read table inputs
  const rows =
document.querySelectorAll('.process-row');
  return Array.from(rows).map((row, index)
=> ({
    pid: `P${index + 1}`,
    arrival:
Number.parseInt(row.querySelector('.arrival
-input')?.value ?? '0', 10) || 0,
    burst:
Number.parseInt(row.querySelector('.burst-
input')?.value ?? '1', 10) || 1,
    priority:
Number.parseInt(row.querySelector('.priorit
y-input')?.value ?? '0', 10) || 0,
  }));
}

function validateInputs(processes,
quantumRequired, quantum) {
  // Clear validation styles
  let isValid = true;
  document.querySelectorAll('.process-
input').forEach(el => el.classList.remove('is-
invalid'));
  document.getElementById('time-
quantum')?.classList.remove('is-invalid');
  processes.forEach((p, index) => {
    const row =
document.getElementById(`row-p${index +
1}`);
    if (!row) return;

```

```

    const arrivalInput =
row.querySelector('.arrival-input');
    const burstInput =
row.querySelector('.burst-input');
    const priorityInput =
row.querySelector('.priority-input');
    if (Number.isNaN(p.arrival) || p.arrival <
0 || p.arrival > MAX_INPUT_VALUE) {
      arrivalInput?.classList.add('is-invalid');
      isValid = false;
    }
    if (Number.isNaN(p.burst) || p.burst < 1 ||
p.burst > MAX_INPUT_VALUE) {
      burstInput?.classList.add('is-invalid');
      isValid = false;
    }
    if (Number.isNaN(p.priority) || p.priority
< 0 || p.priority > MAX_INPUT_VALUE) {
      priorityInput?.classList.add('is-invalid');
      isValid = false;
    }
  });

  if (quantumRequired &&
(Number.isNaN(quantum) || quantum < 1 ||
quantum > MAX_INPUT_VALUE)) {
    document.getElementById('time-
quantum')?.classList.add('is-invalid');
    isValid = false;
  }
  return isValid;
}

function runSimulation() {
  if (!currentAlgorithm) {
    // Missing algorithm
    showNotice('Select an algorithm first.');
```

```

    return;
  }
  const algorithmName =
getAlgorithmName(currentAlgorithm);
  const processes = readProcessInputs();
  const quantum =
Number.parseInt(document.getElementById(
'time-quantum')?.value ?? '2', 10) || 2;
  const needsQuantum = currentAlgorithm
=== 'RR' || currentAlgorithm ===
'PriorityRR';
  if (!validateInputs(processes,
needsQuantum, quantum)) {

```



```

    // Invalid inputs
    showNotice('Fix highlighted fields before
running.');
```

return;

```

  }
  if (typeof window.runSelectedAlgorithm
!== 'function') {
    // No algorithm yet
    showNotice(`No algorithm yet for
${algorithmName}.`);
    return;
  }

  // Delegate to the algorithm bridge
  provided by the selected script.
  let result;
  try {
    result = window.runSelectedAlgorithm({
      algorithm: currentAlgorithm,
      processes,
      quantum,
      priorityMode:
document.querySelector('input[name="priori
ty-mode"]:checked')?.value ?? 'lower-
higher',
      preemptionMode:
document.querySelector('input[name="pree
mption-mode"]:checked')?.value ?? 'non-
preemptive',
    });
  } catch {
    showNotice(`No algorithm yet for
${algorithmName}.`);
    return;
  }
  if (!result ||
!Array.isArray(result.ganttBlocks) ||
!Array.isArray(result.results)) {
    showNotice(`No algorithm yet for
${algorithmName}.`);
    return;
  }
  lastResult = result;
  // Render outputs
  renderGantt(result.ganttBlocks,
result.results);
  renderResultsTable(result.results);
  renderMetrics(result.results);

```

```

    document.getElementById('simulation-
empty')?.classList.add('hidden');
    document.getElementById('gantt-
panel')?.classList.remove('hidden');
    const label =
document.getElementById('gantt-algo-
label');
    if (label) label.textContent =
`${algorithmName} — Gantt Chart`;
  }
  function renderGantt(ganttBlocks, results) {
    const container =
document.getElementById('gantt-
container');
    const timeRow =
document.getElementById('gantt-time');
    if (!container || !timeRow) return;
    // Clear gantt
    container.innerHTML = "";
    timeRow.innerHTML = "";
    const totalTime =
Math.max(...ganttBlocks.map(b => b.end),
1);
    const barsFragment =
document.createDocumentFragment();
    ganttBlocks.forEach(block => {
      // Segment width
      const duration = Math.max(block.end -
block.start, 0);
      const widthPercent = (duration /
totalTime) * 100;
      const bar =
document.createElement('div');
      bar.className = `block
${getPidColorClass(block.pid)}`;
      bar.style.width = `${widthPercent}%`;
      bar.textContent = block.pid;
      barsFragment.appendChild(bar);
    });
    container.appendChild(barsFragment);
    if (window.anime) {
      // Animate width only when the animation
      library is present.
      anime({
        targets: '#gantt-container .block',
        width: ['0%', el => el.style.width],
        easing: 'easeOutQuart',
        duration: 600,
        delay: anime.stagger(100),

```

```

    });
  }
  // Batch time marker inserts.
  const timeFragment =
document.createDocumentFragment();
  ganttBlocks.forEach(block => {
    // Start marker
    const marker =
document.createElement('span');
    marker.textContent = String(block.start);
    marker.style.position = 'absolute';
    marker.style.left = `${(block.start /
totalTime) * 100}%`;
    marker.style.transform = 'translateX(-
50%)';
    timeFragment.appendChild(marker);
  });
  const finalMarker =
document.createElement('span');
  // End marker
  finalMarker.textContent =
String(totalTime);
  finalMarker.style.position = 'absolute';
  finalMarker.style.left = '100%';
  finalMarker.style.transform = 'translateX(-
50%)';
  timeFragment.appendChild(finalMarker);
  timeRow.appendChild(timeFragment);
  renderGanttLegend(results,
ganttBlocks.some(b => b.pid === 'IDLE'));
}
function renderGanttLegend(results, hasIdle)
{
  const legend =
document.getElementById('gantt-legend');
  if (!legend) return;
  legend.innerHTML = "";
  const legendFragment =
document.createDocumentFragment();
  results.forEach(p => {
    const item =
document.createElement('span');
    item.className = 'legend-item';
    const dot =
document.createElement('span');
    dot.className = `legend-dot
${getPidColorClass(p.pid)}`;
    item.appendChild(dot);

```

```

    item.appendChild(document.createTextNode(
p.pid));
    legendFragment.appendChild(item);
  });
  if (hasIdle) {
    const idleItem =
document.createElement('span');
    idleItem.className = 'legend-item';
    const idleDot =
document.createElement('span');
    idleDot.className = 'legend-dot pid-
color-idle';
    idleItem.appendChild(idleDot);
    idleItem.appendChild(document.createTextNode(
'IDLE'));
    legendFragment.appendChild(idleItem);
  }
  legend.appendChild(legendFragment);
}
function renderResultsTable(results) {
  const tbody =
document.getElementById('results-tbody');
  if (!tbody) return;
  tbody.innerHTML = "";
  // Keep result ordering stable by PID
  regardless of algorithm output.
  const displayResults = [...results].sort((a, b)
=> {
    const aNum =
Number.parseInt(String(a.pid).replace(/^(P/i,
"), 10);
    const bNum =
Number.parseInt(String(b.pid).replace(/^(P/i,
"), 10);
    if (Number.isFinite(aNum) &&
Number.isFinite(bNum) && aNum !==
bNum) {
      return aNum - bNum;
    }
    return
String(a.pid).localeCompare(String(b.pid),
undefined, { numeric: true });
  });
  // Result rows
  const rowsFragment =
document.createDocumentFragment();
  displayResults.forEach(p => {
    const row = document.createElement('tr');

```

```

row.appendChild(buildPidResultCell(p.pid))
;
row.appendChild(buildTextCell(p.arrival));
row.appendChild(buildTextCell(p.burst));
row.appendChild(buildTextCell(p.ct,
true));
row.appendChild(buildTextCell(p.tat,
true));
row.appendChild(buildTextCell(p.wt,
true));
rowsFragment.appendChild(row);
});
tbody.appendChild(rowsFragment);
}
function buildPidResultCell(pid) {
const td = document.createElement('td');
const pill =
document.createElement('span');
pill.className = 'pid-colored-pill';
const dot =
document.createElement('span');
dot.className = `pid-color-dot
${getPidColorClass(pid)}`;
pill.appendChild(dot);
pill.appendChild(document.createTextNode(
pid));
td.appendChild(pill);
return td;
}
function buildTextCell(value, center = false)
{
const td = document.createElement('td');
if (center) td.className = 'text-center';
td.textContent = value === null || value
=== undefined ? '-' : String(value);
return td;
}
function renderMetrics(results) {
// Metric placeholders
const hasComputedMetrics =
results.every(p => Number.isFinite(p.wt)
&& Number.isFinite(p.tat));
if (!hasComputedMetrics) {
setText('avg-wt-display', '-');
setText('avg-tat-display', '-');
return;
}
const avgWt = results.reduce((sum, p) =>
sum + p.wt, 0) / results.length;

```

```

const avgTat = results.reduce((sum, p) =>
sum + p.tat, 0) / results.length;
setText('avg-wt-display',
avgWt.toFixed(2));
setText('avg-tat-display',
avgTat.toFixed(2));
}
function replayAnimation() {
if (!lastResult) return;
const btn =
document.getElementById('btn-replay');
// Trigger a short CSS animation to indicate
replay.
btn?.classList.add('btn-replaying');
setTimeout(() =>
btn?.classList.remove('btn-replaying'), 500);
renderGantt(lastResult.ganttBlocks,
lastResult.results);
}
function resetAll() {
// Reset template
if (tableBody) tableBody.innerHTML = "";
processCount = 0;
for (let i = 0; i < MIN_PROCESSES; i +=
1) {
addProcess();
}
setAlgorithm("");
setText('avg-wt-display', '-');
setText('avg-tat-display', '-');
const legend =
document.getElementById('gantt-legend');
if (legend) legend.innerHTML = "";
const tbody =
document.getElementById('results-tbody');
if (tbody) {
tbody.innerHTML = `
<tr>
<td>-</td>
<td class="text-center">-</td>
<td class="text-center">-</td>
<td class="text-center">-</td>
<td class="text-center">-</td>
<td class="text-center">-</td>
</tr>
`;
}
}

```

```

    document.getElementById('simulation-
empty')?.classList.remove('hidden');
    document.getElementById('gant-
panel')?.classList.add('hidden');
    lastResult = null;
}
function getPidColorClass(pid) {
    // PID color slot
    if (pid === 'IDLE') return 'pid-color-idle';
    const pidNumber =
Number.parseInt(String(pid).replace(/^[P/i,
"), 10);
    if (!Number.isFinite(pidNumber) ||
pidNumber < 1) return 'pid-color-idle';
    const slot = ((pidNumber - 1) % 10) + 1;
    return `pid-color-${slot}`;
}
function setText(id, value) {
    const el = document.getElementById(id);
    if (el) el.textContent = value;
}
function showNotice(message, tone = 'error',
duration = 2600) {
    // Snackbar message
    if (!snackbarEl) return;
    if (snackbarHideTimer) {
        clearTimeout(snackbarHideTimer);
    }
    snackbarEl.textContent = message;
    snackbarEl.classList.remove('show',
'snackbar-error', 'snackbar-info');
    snackbarEl.classList.add(tone === 'info' ?
'snackbar-info' : 'snackbar-error');
    requestAnimationFrame(() => {
        snackbarEl.classList.add('show');
    });
    snackbarHideTimer = setTimeout(() => {
        snackbarEl.classList.remove('show');
    }, duration);
}

function getAlgorithmName(code) {
    return ALGORITHM_NAMES[code] ??
code;
}
/​assets/js/algorithms/fcfs.js
function runFCFS(processes) {
    // FCFS scheduling with idle gaps when no
process is ready.

```

```

    const sorted = [...processes].sort((a, b) => {
        if (a.arrival !== b.arrival) return a.arrival -
b.arrival;
        return a.pid.localeCompare(b.pid,
undefined, { numeric: true });
    });
    let currentTime = 0;
    const ganttBlocks = [];
    const results = [];
    for (const process of sorted) {
        if (currentTime < process.arrival) {
            ganttBlocks.push({
                pid: 'IDLE',
                start: currentTime,
                end: process.arrival,
            });
            currentTime = process.arrival;
        }
        const startTime = currentTime;
        const completionTime = currentTime +
process.burst;
        const turnaround = completionTime -
process.arrival;
        const waiting = turnaround -
process.burst;
        ganttBlocks.push({
            pid: process.pid,
            start: startTime,
            end: completionTime,
        });
        results.push({
            pid: process.pid,
            arrival: process.arrival,
            burst: process.burst,
            ct: completionTime,
            tat: turnaround,
            wt: waiting,
        });
        currentTime = completionTime;
    }
    return { ganttBlocks, results };
}
/* bridge: main.js */
const previousRunSelectedAlgorithmFCFS
= window.runSelectedAlgorithm;
window.runSelectedAlgorithm = function
runSelectedAlgorithm(input) {
    // Chain algorithm handlers in load order.
    if (input.algorithm === 'FCFS') {

```

```

    const mapped = input.processes.map((p)
=> ({
    pid: p.pid,
    arrival: p.arrival,
    burst: p.burst,
    priority: p.priority,
    }));
    return runFCFS(mapped);
  }
  if (typeof
previousRunSelectedAlgorithmFCFS ===
'function') {
    return
previousRunSelectedAlgorithmFCFS(input);
  }
  throw new Error('No algorithm yet');
};
/assets/js/algorithms/sjf.js
function runSJF(processes) {
  // Non-preemptive SJF with explicit idle
  periods.
  const pending = processes.map((process)
=> ({ ...process, done: false }));
  const ganttBlocks = [];
  const results = [];
  let currentTime = 0;
  let completed = 0;
  while (completed < pending.length) {
    // Re-evaluate the ready queue at each
    time step.
    const ready = pending.filter(
      (process) => !process.done &&
process.arrival <= currentTime
    );
    if (ready.length === 0) {
      const nextArrival = Math.min(
        ...pending
        .filter((process) => !process.done)
        .map((process) => process.arrival)
      );
      if (currentTime < nextArrival) {
        ganttBlocks.push({
          pid: 'IDLE',
          start: currentTime,
          end: nextArrival,
        });
        currentTime = nextArrival;
      }
      continue;

```

```

    }
    ready.sort((a, b) => {
      if (a.burst !== b.burst) return a.burst -
b.burst;
      if (a.arrival !== b.arrival) return a.arrival
- b.arrival;
      return a.pid.localeCompare(b.pid,
undefined, { numeric: true });
    });
    const selected = ready[0];
    const startTime = currentTime;
    const completionTime = startTime +
selected.burst;
    const turnaround = completionTime -
selected.arrival;
    const waiting = turnaround -
selected.burst;

    ganttBlocks.push({
      pid: selected.pid,
      start: startTime,
      end: completionTime,
    });

    results.push({
      pid: selected.pid,
      arrival: selected.arrival,
      burst: selected.burst,
      ct: completionTime,
      tat: turnaround,
      wt: waiting,
    });

    selected.done = true;
    completed += 1;
    currentTime = completionTime;
  }

  return { ganttBlocks, results };
}

/* bridge: main.js */
const previousRunSelectedAlgorithmSJF =
window.runSelectedAlgorithm;

window.runSelectedAlgorithm = function
runSelectedAlgorithm(input) {
  // Chain algorithm handlers in load order.
  if (input.algorithm === 'SJF') {

```

```

    const mapped = input.processes.map((p)
=> ({
    pid: p.pid,
    arrival: p.arrival,
    burst: p.burst,
    priority: p.priority,
    }));
    return runSJF(mapped);
  }
  if (typeof
previousRunSelectedAlgorithmSJF ===
'function') {
    return
previousRunSelectedAlgorithmSJF(input);
  }
  throw new Error('No algorithm yet');
};
/assets/js/algorithms/srt.js
function runSRT(processes){
  // Preemptive SRT schedules in single time
  quanta.
  const ordered=[...processes].sort((a,b)=>{
    if(a.arrival!==b.arrival)return a.arrival-
b.arrival;
    return
a.pid.localeCompare(b.pid,undefined,{nume
ric:true});
  });
  const remaining=new
Map(ordered.map((p)=>[p.pid,p.burst]));
  const completion=new Map();
  const ganttBlocks=[];
  let currentTime=0;
  let completed=0;
  // Coalesce adjacent blocks for the same
  PID.
  const appendBlock=(pid,start,end)=>{
    if(end<=start)return;
    const
last=ganttBlocks[ganttBlocks.length-1];

if(last&&last.pid===pid&&last.end===start
){
  last.end=end;
  return;
}
  ganttBlocks.push({pid,start,end});
};
  while(completed<ordered.length){

```

```

    // Select from processes that have arrived
    and still need CPU time.
    const
ready=ordered.filter((p)=>p.arrival<=current
Time&&(remaining.get(p.pid)??0)>0);
    if(ready.length===0){
      const nextArrival=Math.min(
        ...ordered
        .filter((p)=>(remaining.get(p.pid)??0)>0)
        .map((p)=>p.arrival)
      );
      appendBlock('IDLE',currentTime,nextArriva
l);
      currentTime=nextArrival;
      continue;
    }
    ready.sort((a,b)=>{
      const
remainingA=remaining.get(a.pid)??0;
      const
remainingB=remaining.get(b.pid)??0;
      if(remainingA!==remainingB)return
remainingA-remainingB;
      if(a.arrival!==b.arrival)return a.arrival-
b.arrival;
      return
a.pid.localeCompare(b.pid,undefined,{nume
ric:true});
    });
    const selected=ready[0];
    appendBlock(selected.pid,currentTime,curre
ntTime+1);
    currentTime+=1;
    const
newRemaining=(remaining.get(selected.pid)
??0)-1;
    remaining.set(selected.pid,newRemaining);
    if(newRemaining===0){
      completion.set(selected.pid,currentTime);
      completed+=1;
    }
  }
  const results=processes.map((p)=>{
    const ct=completion.get(p.pid)??0;
    const tat=ct-p.arrival;
    const wt=tat-p.burst;
    return{
      pid:p.pid,
      arrival:p.arrival,

```

```

    burst:p.burst,
    ct,
    tat,
    wt,
  });
});
return {ganttBlocks, results};
}

/* bridge: main.js */
const
previousRunSelectedAlgorithmSRT=window.runSelectedAlgorithm;

window.runSelectedAlgorithm=function
runSelectedAlgorithm(input){
  // Chain algorithm handlers in load order.
  if(input.algorithm==='SRT'){
    const
mapped=input.processes.map((p)=>({
  pid:p.pid,
  arrival:p.arrival,
  burst:p.burst,
  priority:p.priority,
})));
    return runSRT(mapped);
  }
  if(typeof
previousRunSelectedAlgorithmSRT==='function'){
    return
previousRunSelectedAlgorithmSRT(input);
  }
  throw new Error('No algorithm yet');
};

/assets/js/algorithms/rr.js
function runRR(processes, quantum){
  // Round-robin scheduling with a fixed
time slice.
  const
q=Number.isFinite(quantum)&&quantum>0
?Math.floor(quantum):1;
  const ordered=[...processes].sort((a,b)=>{
    if(a.arrival!==b.arrival)return a.arrival-
b.arrival;
    return
a.pid.localeCompare(b.pid, undefined, {numeric:true});
  });

```

```

    const remaining=new
Map(ordered.map((p)=>[p.pid,p.burst]));
    const completion=new Map();
    const queue=[];
    const ganttBlocks=[];

    let currentTime=0;
    let completed=0;
    let nextArrivalIndex=0;

    // Coalesce adjacent blocks for the same
PID.
    const appendBlock=(pid,start,end)=>{
      if(end<=start)return;
      const
last=ganttBlocks[ganttBlocks.length-1];

      if(last&&last.pid===pid&&last.end===start
){
        last.end=end;
        return;
      }
      ganttBlocks.push({pid,start,end});
    };
    // Enqueue arrivals that have reached the
current time.
    const enqueueArrivals=()=>{
      while(nextArrivalIndex<ordered.length&&ordered[nextArrivalIndex].arrival<=currentTime){
        queue.push(ordered[nextArrivalIndex]);
        nextArrivalIndex+=1;
      }
    };
    while(completed<ordered.length){
      enqueueArrivals();
      if(queue.length===0){
        const
nextArrival=ordered[nextArrivalIndex]?arrival;
        if(nextArrival===undefined)break;
        appendBlock('IDLE',currentTime,nextArrival);
        currentTime=nextArrival;
        enqueueArrivals();
        continue;
      }
      const current=queue.shift();

```

```

    if(!current)continue;
    const
remainingBefore=remaining.get(current.pid)
??0;
    if(remainingBefore<=0)continue;
    // Run for either the quantum or remaining
burst, whichever is smaller.
    const
runFor=Math.min(q,remainingBefore);
    const start=currentTime;
    const end=currentTime+runFor;
    appendBlock(current.pid,start,end);
    currentTime=end;
    remaining.set(current.pid,remainingBefore-
runFor);
    enqueueArrivals();
    if((remaining.get(current.pid)??0)>0){
        queue.push(current);
    }else{
        completion.set(current.pid,currentTime);
        completed+=1;
    }
}
const results=processes.map((p)=>{
    const ct=completion.get(p.pid)??0;
    const tat=ct-p.arrival;
    const wt=tat-p.burst;
    return {
        pid:p.pid,
        arrival:p.arrival,
        burst:p.burst,
        ct,
        tat,
        wt,
    };
});
return {ganttBlocks,results};
}
/* bridge: main.js */
const
previousRunSelectedAlgorithmRR=window
.runSelectedAlgorithm;
window.runSelectedAlgorithm=function
runSelectedAlgorithm(input){
    // Chain algorithm handlers in load order.
    if(input.algorithm==='RR'){
        const
mapped=input.processes.map((p)=>({
            pid:p.pid,

```

```

            arrival:p.arrival,
            burst:p.burst,
            priority:p.priority,
        }));
        return runRR(mapped,input.quantum);
    }
    if(typeof
previousRunSelectedAlgorithmRR==='funct
ion'){
        return
previousRunSelectedAlgorithmRR(input);
    }
    throw new Error('No algorithm yet');
};
/assets/js/algorithms/priority.js
function
comparePriorityValue(a,b,priorityMode){
    // Normalize priority ordering based on
user convention.
    if(priorityMode==='higher-higher'){
        return b-a;
    }
    return a-b;
}
function
runPriority(processes,priorityMode,preempti
onMode){
    // Priority scheduling with optional
preemption.
    const ordered=[...processes].sort((a,b)=>{
        if(a.arrival!==b.arrival)return a.arrival-
b.arrival;
        return
a.pid.localeCompare(b.pid,undefined,{nume
ric:true});
    });
    const remaining=new
Map(ordered.map((p)=>[p.pid,p.burst]));
    const completion=new Map();
    const gantBlocks=[];
    let currentTime=0;
    let completed=0;
    // Coalesce adjacent blocks for the same
PID.
    const appendBlock=(pid,start,end)=>{
        if(end<=start)return;
        const
last=gantBlocks[gantBlocks.length-1];

```



```

if(last&&last.pid===pid&&last.end===start
){
    last.end=end;
    return;
}
gantBlocks.push( {pid,start,end});
};
// Select the best candidate from currently
ready processes.
const selectReady=()=>{
    const
ready=ordered.filter((p)=>p.arrival<=current
Time&&(remaining.get(p.pid)??0)>0);
    if(ready.length===0)return null;
    ready.sort((a,b)=>{
        const
byPriority=comparePriorityValue(a.priority,
b.priority,priorityMode);
        if(byPriority!==(0))return byPriority;
        if(a.arrival!==(b.arrival))return a.arrival-
b.arrival;
        return
a.pid.localeCompare(b.pid,undefined,{nume
ric:true});
    });
    return ready[0];
};
while(completed<ordered.length){
    const selected=selectReady();
    if(!selected){
        const nextArrival=Math.min(
            ...ordered
.filter((p)=>(remaining.get(p.pid)??0)>0)
.map((p)=>p.arrival)
        );
    appendBlock('IDLE',currentTime,nextArriva
l);
        currentTime=nextArrival;
        continue;
    }
    // Preemptive mode runs in single time
units.
    if(preemptionMode==='preemptive'){
    appendBlock(selected.pid,currentTime,curre
ntTime+1);
        currentTime+=1;

```

```

        const
newRemaining=(remaining.get(selected.pid)
??0)-1;
        remaining.set(selected.pid,newRemaining);
        if(newRemaining===0){
            completion.set(selected.pid,currentTime);
            completed+=1;
        }
        continue;
    }
    const
runFor=remaining.get(selected.pid)??0;
    appendBlock(selected.pid,currentTime,curre
ntTime+runFor);
        currentTime+=runFor;
        remaining.set(selected.pid,0);
        completion.set(selected.pid,currentTime);
        completed+=1;
    }
    const results=processes.map((p)=>{
        const ct=completion.get(p.pid)??0;
        const tat=ct-p.arrival;
        const wt=tat-p.burst;
        return{
            pid:p.pid,
            arrival:p.arrival,
            burst:p.burst,
            ct,
            tat,
            wt,
        };
    });
    return {gantBlocks,results};
}
/* bridge: main.js */
const
previousRunSelectedAlgorithmPriority=win
dow.runSelectedAlgorithm;
window.runSelectedAlgorithm=function
runSelectedAlgorithm(input){
    // Chain algorithm handlers in load order.
    if(input.algorithm==='Priority'){
        const
mapped=input.processes.map((p)=>({
            pid:p.pid,
            arrival:p.arrival,
            burst:p.burst,
            priority:p.priority,
        }));

```

```

    return
runPriority(mapped,input.priorityMode,input
t.preemptionMode);
}
if(typeof
previousRunSelectedAlgorithmPriority==='f
unction'){
    return
previousRunSelectedAlgorithmPriority(inpu
t);
}
throw new Error('No algorithm yet');
};
/assets/js/algorithms/priority-rr.js
function
comparePriorityValuePriorityRR(a,b,priorit
yMode){
    // Normalize priority ordering based on
user convention.
    if(priorityMode==='higher-higher'){
        return b-a;
    }
    return a-b;
}
function
runPriorityRR(processes,quantum,priorityM
ode){
    // Priority queues with round-robin time
slicing per priority.
    const
q=Number.isFinite(quantum)&&quantum>0
?Math.floor(quantum):1;
    const ordered=[...processes].sort((a,b)=>{
        if(a.arrival!==b.arrival)return a.arrival-
b.arrival;
        return
a.pid.localeCompare(b.pid,undefined,{nume
ric:true});
    });
    const remaining=new
Map(ordered.map((p)=>[p.pid,p.burst]));
    const completion=new Map();
    const queueByPriority=new Map();
    const inQueue=new Set();
    const ganttBlocks=[];

    let currentTime=0;
    let completed=0;
    let nextArrivalIndex=0;

```

```

    // Coalesce adjacent blocks for the same
PID.
    const appendBlock=(pid,start,end)=>{
        if(end<=start)return;
        const
last=ganttBlocks[ganttBlocks.length-1];
        if(last&&last.pid===pid&&last.end===start
){
            last.end=end;
            return;
        }
        ganttBlocks.push({pid,start,end});
    };
    // Lazily create per-priority queues.
    const getQueue=(priority)=>{
        if(!queueByPriority.has(priority)){
            queueByPriority.set(priority,[]);
        }
        return queueByPriority.get(priority);
    };
    // Enqueue arrivals once; inQueue prevents
duplicates.
    const enqueueArrivals=()=>{
        while(nextArrivalIndex<ordered.length&&o
rdered[nextArrivalIndex].arrival<=currentTi
me){
            const
process=ordered[nextArrivalIndex];

            if(!inQueue.has(process.pid)&&(remaining.
get(process.pid)??0)>0){

                getQueue(process.priority).push(process);
                inQueue.add(process.pid);
            }
            nextArrivalIndex+=1;
        }
    };
    // Pick highest-priority non-empty queue.
    const pickBestPriority=()=>{
        const activePriorities=[];
        queueByPriority.forEach((queue,priority)=>
{
            if(queue.length>0)activePriorities.push(prior
ity);
        });
        if(activePriorities.length===0)return null;

```

```

activePriorities.sort((a,b)=>comparePriority
ValuePriorityRR(a,b,priorityMode));
return activePriorities[0];
};
// Check if a higher-priority queue is ready
to preempt.
const
hasHigherPriorityReady=(priority)=>{
const bestPriority=pickBestPriority();
if(bestPriority===null)return false;
return
comparePriorityValuePriorityRR(bestPriorit
y,priority,priorityMode)<0;
};
while(completed<ordered.length){
enqueueArrivals();
const activePriority=pickBestPriority();
if(activePriority===null){
const
nextArrival=ordered[nextArrivalIndex]?.arri
val;
if(nextArrival===undefined)break;
appendBlock('IDLE',currentTime,nextArriva
l);
currentTime=nextArrival;
enqueueArrivals();
continue;
}
const queue=getQueue(activePriority);
const current=queue.shift();
if(!current)continue;
inQueue.delete(current.pid);
const
remainingBefore=remaining.get(current.pid)
??0;
if(remainingBefore<=0)continue;
let ran=0;
const
maxSlice=Math.min(q,remainingBefore);
// Step through the slice to allow higher-
priority arrivals to preempt.
while(ran<maxSlice&&(remaining.get(curre
nt.pid)??0)>0){
appendBlock(current.pid,currentTime,curren
tTime+1);
currentTime+=1;
ran+=1;
remaining.set(current.pid,(remaining.get(cur
rent.pid)??0)-1);

```

```

enqueueArrivals();
if((remaining.get(current.pid)??0)>0&&has
HigherPriorityReady(activePriority)){
break;
}
}
if((remaining.get(current.pid)??0)>0){
getQueue(current.priority).push(current);
inQueue.add(current.pid);
}else{
completion.set(current.pid,currentTime);
completed+=1;
}
}
const results=processes.map((p)=>{
const ct=completion.get(p.pid)??0;
const tat=ct-p.arrival;
const wt=tat-p.burst;
return {
pid:p.pid,
arrival:p.arrival,
burst:p.burst,
ct,
tat,
wt,
};
});
return {gantBlocks,results};
}
/* bridge: main.js */
const
previousRunSelectedAlgorithmPriorityRR=
window.runSelectedAlgorithm;
window.runSelectedAlgorithm=function
runSelectedAlgorithm(input){
// Chain algorithm handlers in load order.
if(input.algorithm==='PriorityRR'){
const
mapped=input.processes.map((p)=>({
pid:p.pid,
arrival:p.arrival,
burst:p.burst,
priority:p.priority,
}));
return
runPriorityRR(mapped,input.quantum,input.
priorityMode);
}

```

```
    if(typeof
previousRunSelectedAlgorithmPriorityRR=
=='function'){
    return
previousRunSelectedAlgorithmPriorityRR(i
nput);
    }
    throw new Error('No algorithm yet');
};
```